

计算机

组成原理与系统结构

主讲人：王蓝菁 | 学院：信息工程学院 | 时间：

第六章 指令流水线

When the night is darkest, dawn is nearest. Nothing is impossible for a willing heart.

本章主要内容

- 6.1 流水线概述
- 6.2 流水线处理器的实现
- 6.3 流水线冒险及其处理



MIPS CPU实现方案

■ 单周期方案

- 性能受限于最慢的指令
- 结构简单，实现容易

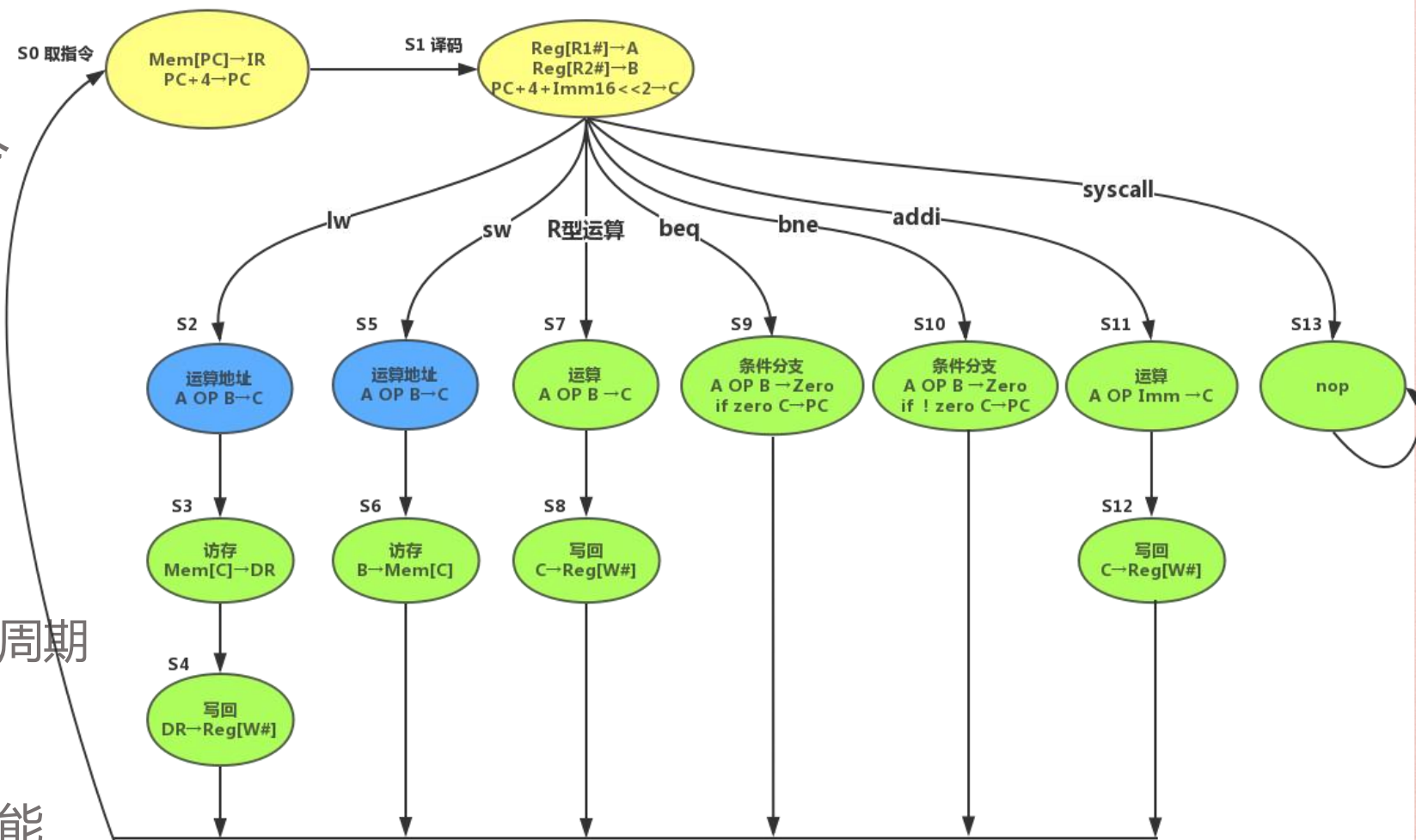
■ 多周期方案

□ 传统多周期

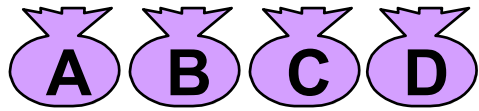
- ◆ 提升性能，复用器件
- ◆ 异步控制，变长指令周期

□ 指令流水线

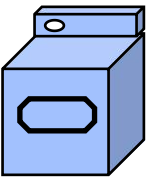
- ◆ 多指令并行，提升性能
- ◆ 部件并发



一个日常生活中的例子—洗衣服



30

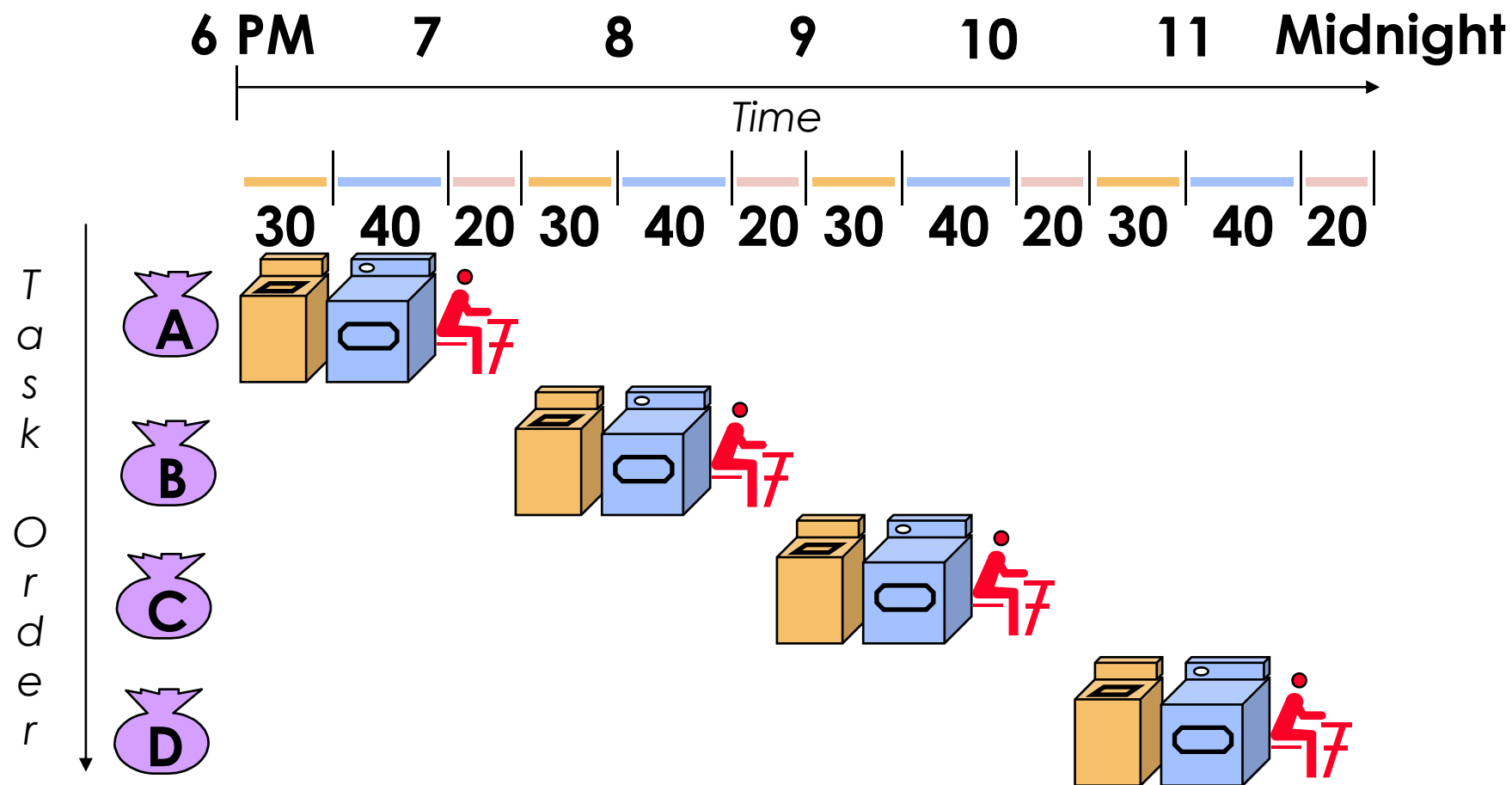


40



20

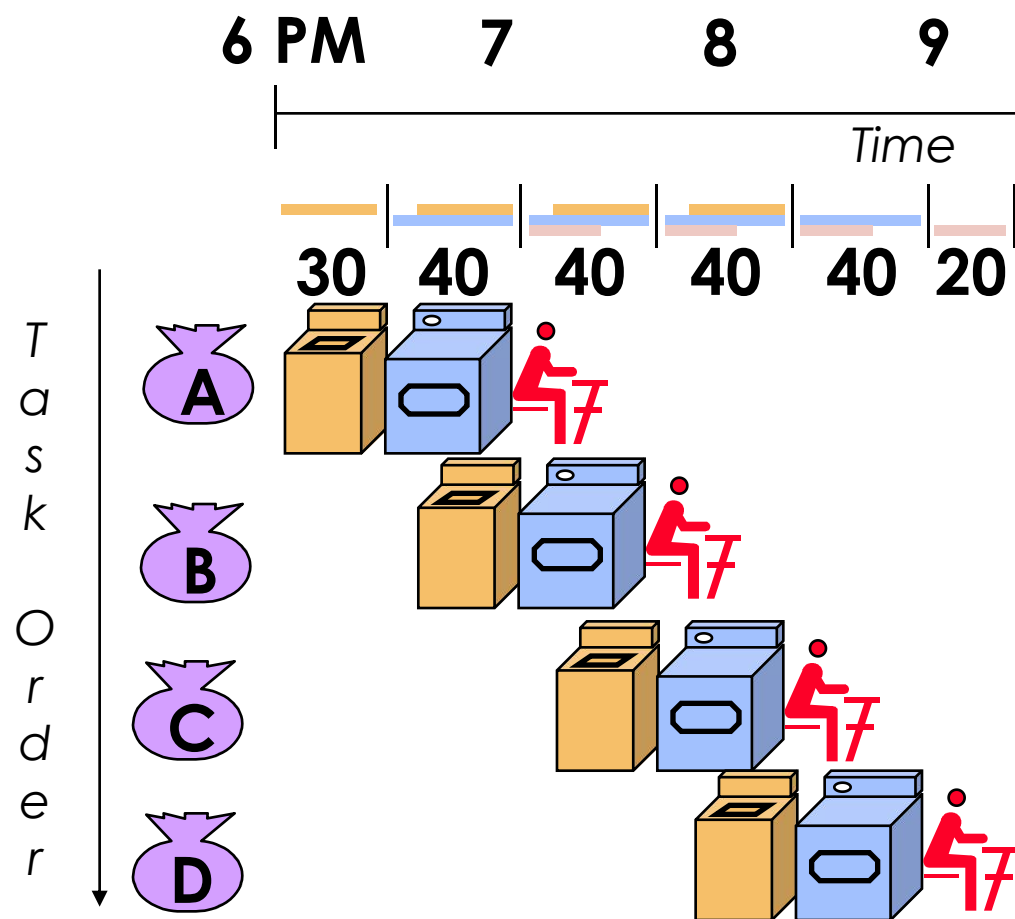
一个日常生活中的例子—洗衣服



■ **串行方式:** 4 批衣服需要花费 6 小时 ($4 \times (30 + 40 + 20) = 360$ 分钟)

■ N批衣服, 需花费的时间为 $N \times (30 + 40 + 20) = 90N$

一个日常生活中的例子—洗衣服



- **流水线方式：**需 $30+4\times 40+20=210$ 分
(3.5小时)
- 如果有N批衣服呢？
- $30+N\times 40+20$ 分钟
- 流水方式下，所用时间主要与最长阶段的时间有关。

指令流水线设计的基本思想

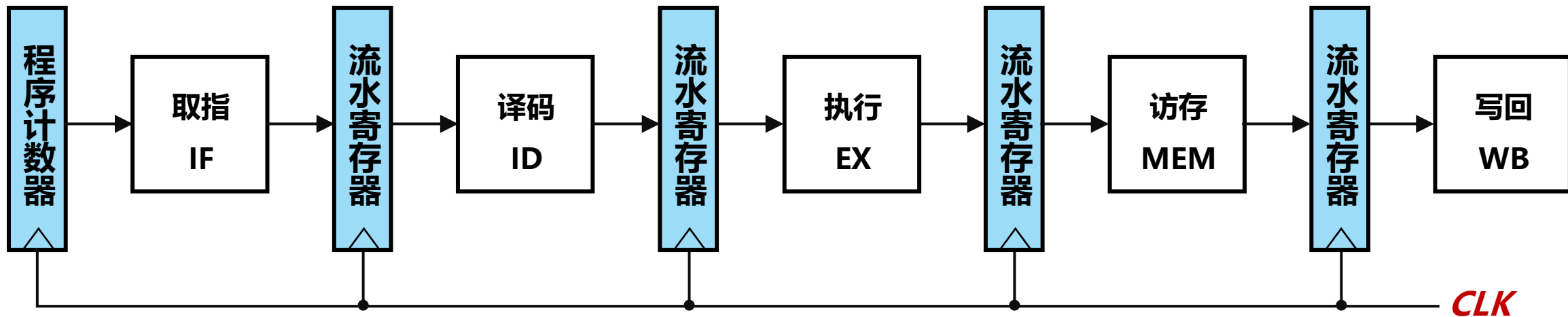
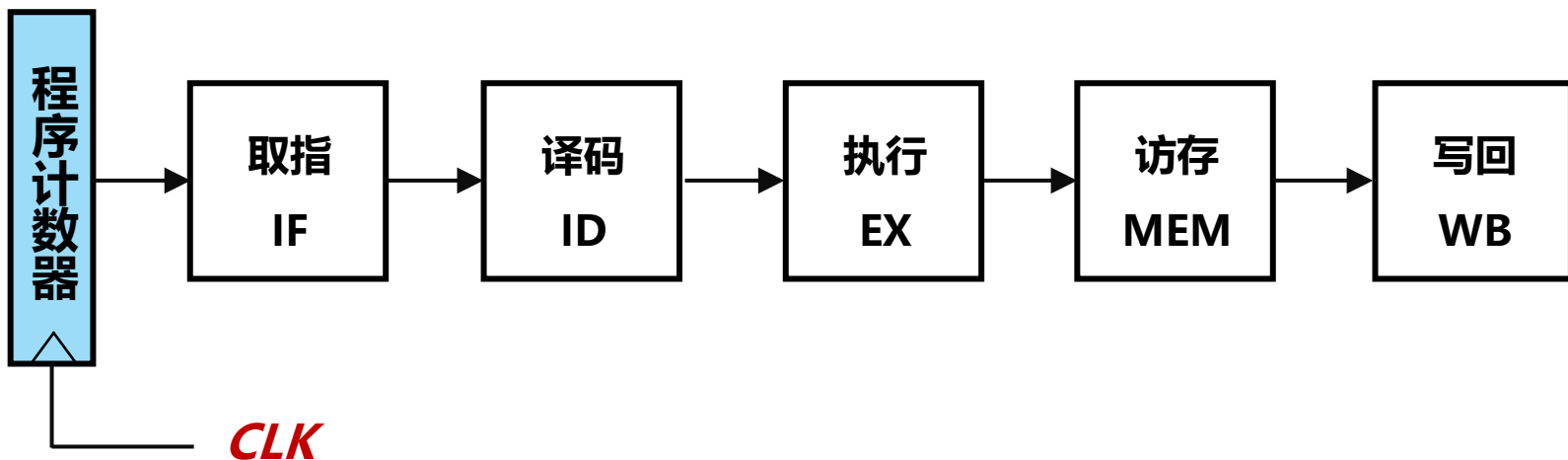
- 将每条指令的执行规整化为若干个同样的流水阶段，每个流水阶段的执行时间一样，都等于一个时钟周期。
- 流水线方式下，一条指令的执行过程被分成若干个操作子过程，每个子过程由一个独立的功能部件来完成。
- 在同一条流水线中，每条指令所包含的操作子过程个数必须一样，每个子过程所花的时间也要相同。
- 一般按**最复杂的指令**来设计流水段个数，以**最复杂的子过程**来设计流水段的宽度。

Load指令的5个阶段

阶段1	阶段2	阶段 3	阶段 4	阶段5
Ifetch	Reg/Dec	Exec	Mem	Wr

- Ifetch (取指) : 取指令并计算PC+4 (指令存储器、Adder)
- Reg/Dec (取数和译码) : 取数同时译码 (寄存器堆读口、指令译码器)
- Exec (执行) : 计算内存单元地址 (扩展器、ALU)
- Mem (读存储器) : 从数据存储器中读 (数据存储器)
- Wr(写寄存器) : 将数据写到寄存器中 (寄存器堆写口)

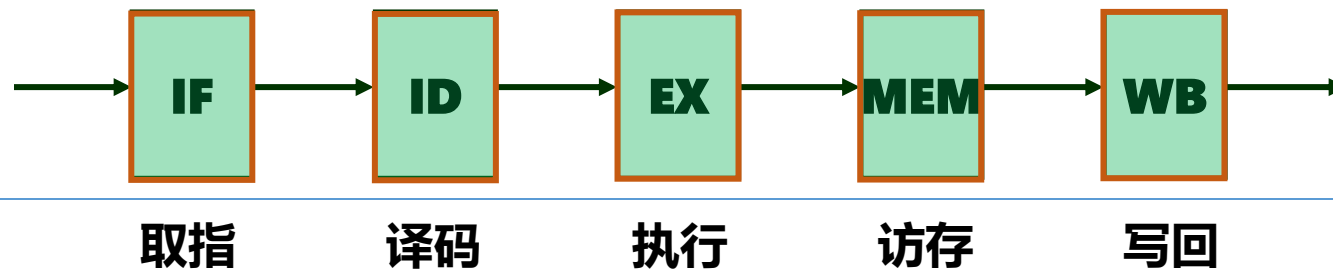
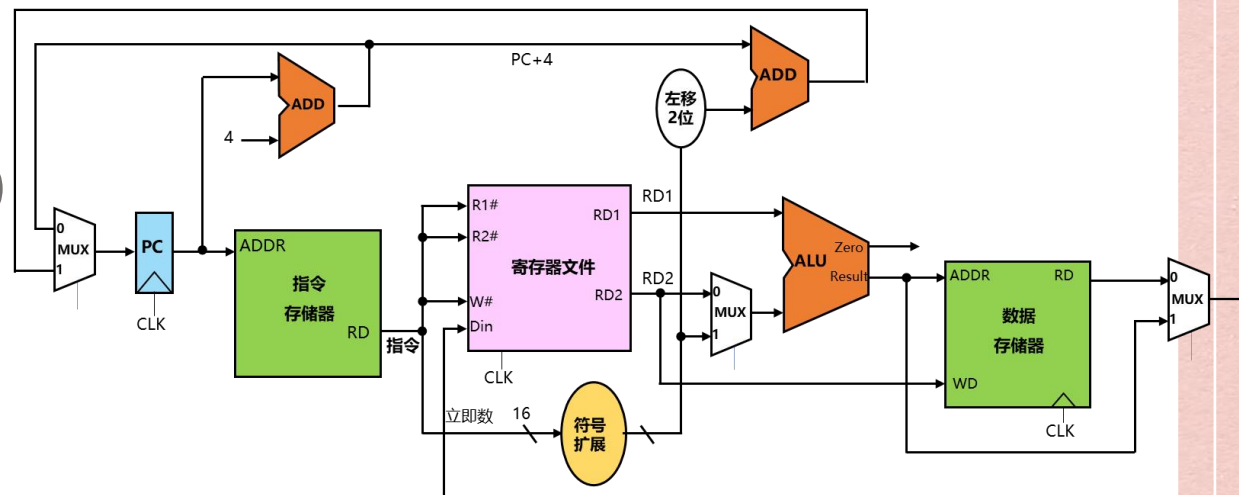
MIPS单周期与流水线



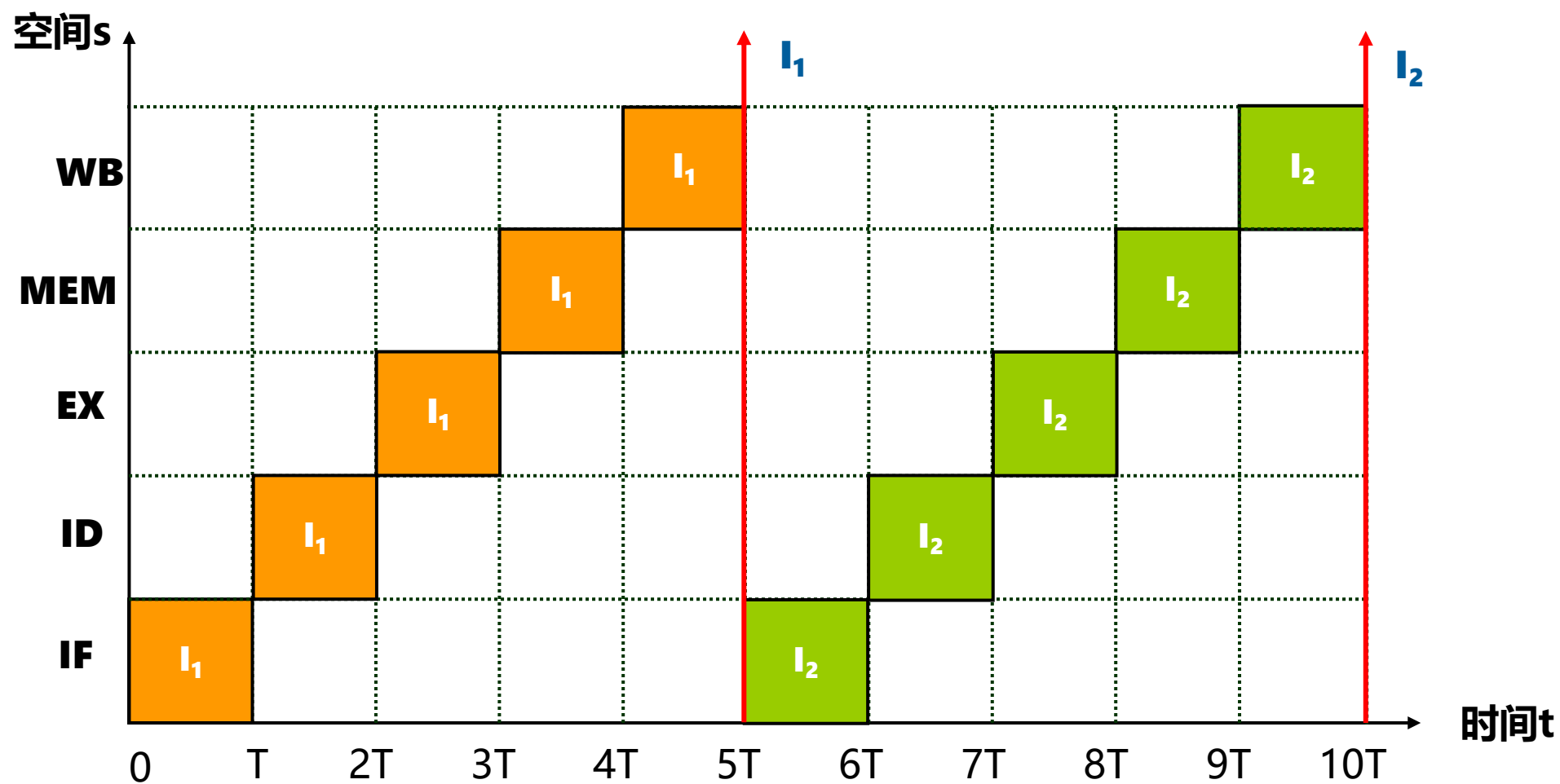
单周期指令运行动态

■ 数据通路细分为5段，总时长为1个时钟周期=5T

- 取指令 IF (Instruction Fetch)
- 指令译码 ID (Instruction Decode)
- 执行运算 EX (Execution)
- 访存阶段 MEM
- 结果写回 WB (Write Back)



单周期时空图

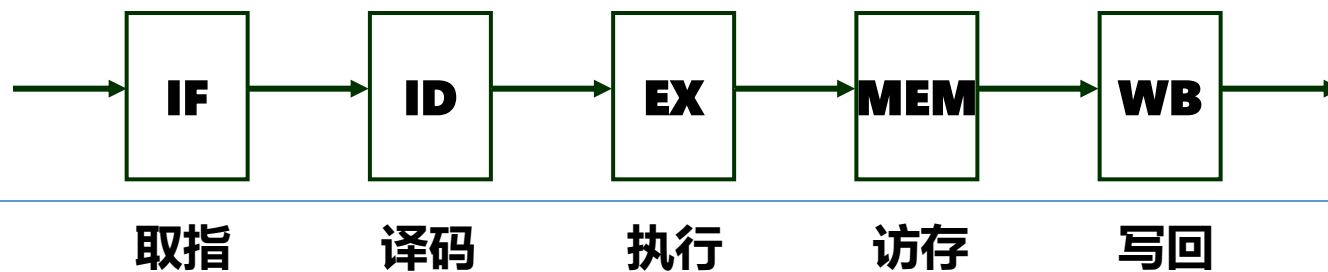
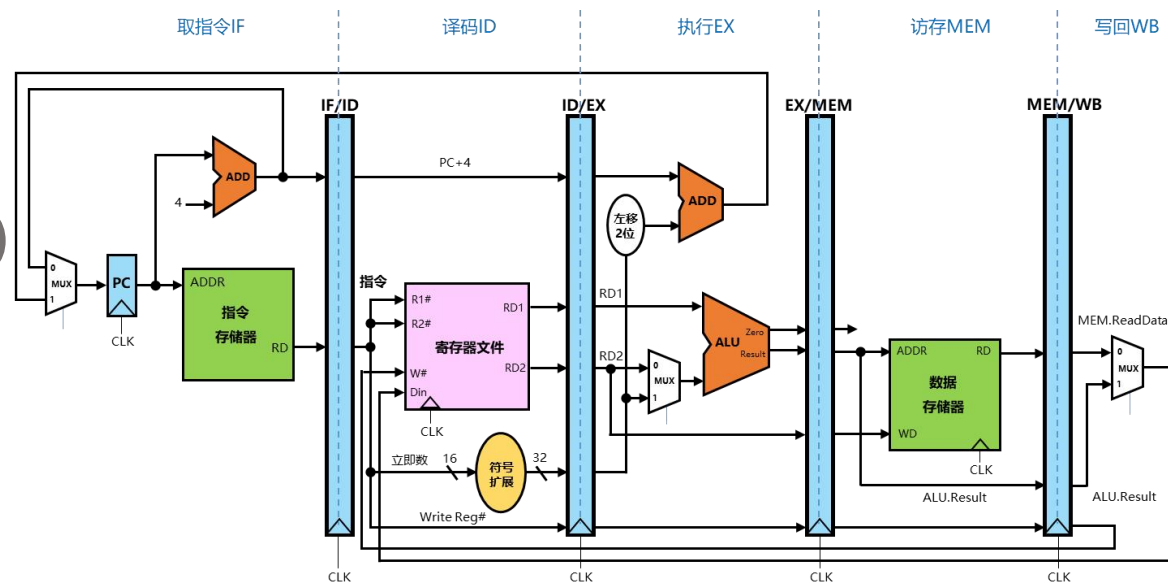


完成n条指令需要 $5nT$

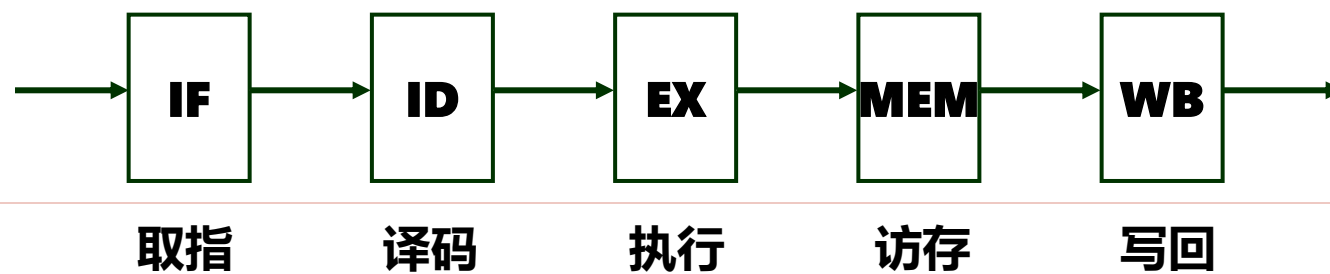
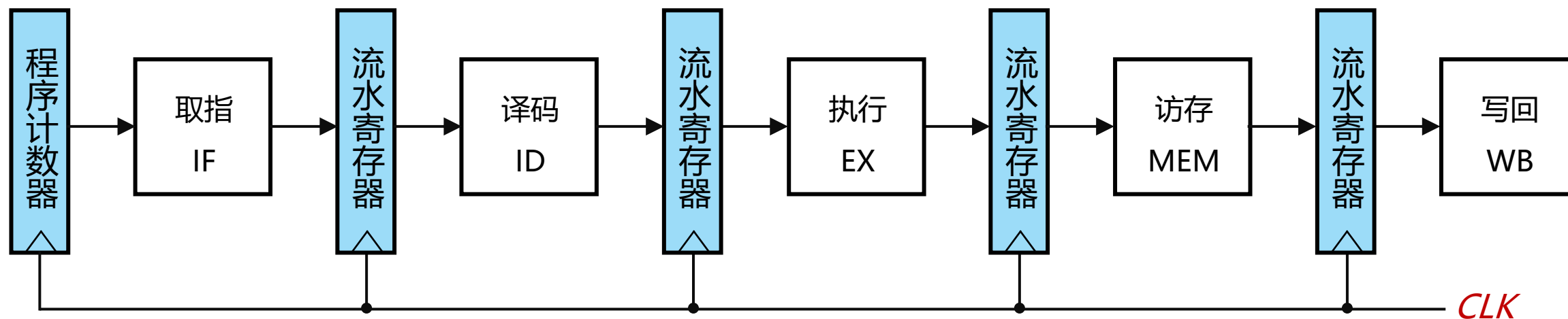
流水线指令运行动态

■ 数据通路细分为5段，各段完全并发

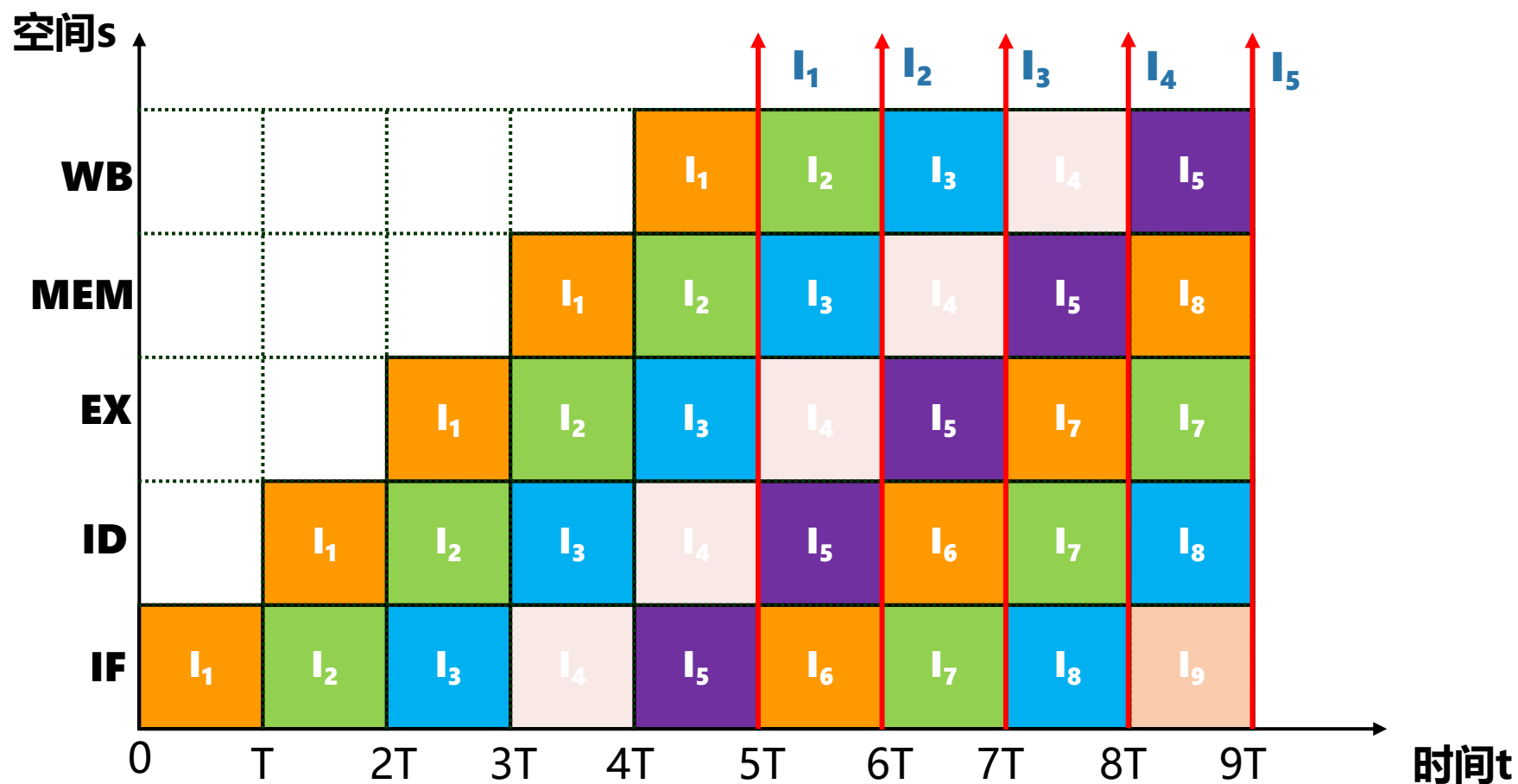
- 取指令 IF (Instruction Fetch)
- 指令译码 ID (Instruction Decode)
- 执行运算 EX (Execution)
- 访存阶段 MEM
- 结果写回 WB (Write Back)



流水线指令运行动态



指令流水线时空图



完成 n 条指令的时间=完成第一条指令时间 $5T + (n-1)*T = (n+4)T$

单周期指令模型与流水模型的性能比较

■ 假定以下每步操作所花时间为：

- 取指：2ns
- 寄存器读：1ns
- ALU操作：2ns
- 存储器读：2ns
- 寄存器写：1ns

单周期指令模型与流水模型的性能比较

■ 单周期模型性能

- 每条指令在一个时钟周期内完成
- 时钟周期等于最长的lw指令的执行时间，即：8ns
- 串行执行时，N条指令的执行时间为：8Nns

■ 流水线模型性能

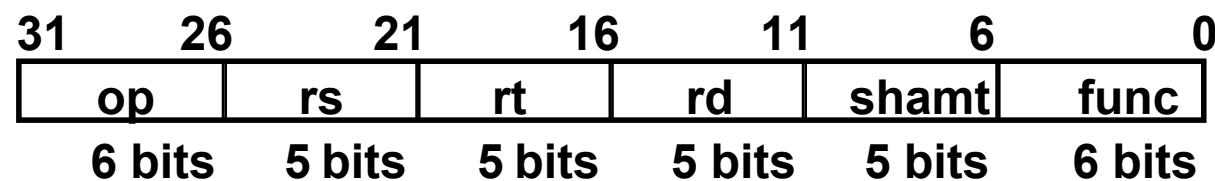
- 时钟周期等于最长阶段所花时间为：2ns
- 每条指令的执行时间为：2nsx5=10ns
- N条指令的执行时间为：(4+N)x2ns
- 在N很大时，比串行方式提高约 4 倍
- 若各阶段操作均衡(例如，各阶段都是2ns)，则提高倍数为5倍。
- 流水线方式下，单条指令执行时间不能缩短，但能大大提高指令吞吐率

理想流水线特征

- **阶段数相同：**所有加工对象均通过同样的工序（阶段）不同指令阶段数不同
 - **段时延相同：**各段传输延迟一致，不能有等待现象，取最慢的同步取指，访存段最慢
 - **无资源冲突：**不同阶段之间无共享资源，各段完全并发取指令、取数存在内存争用
 - **无段间互锁：**进入流水线的对象不受其他阶段的影响多条指令间存在相关和依赖
- ◆ **Microprocessor without interlocked piped stages (MIPS)**

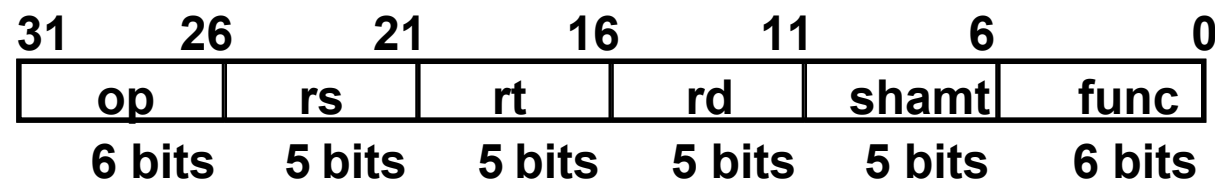
流水线指令集的设计

- **长度尽量一致，有利于简化取指令和指令译码操作**
 - MIPS指令32位，下址计算方便: $PC+4$
 - X86指令从1字节到17字节不等，使取指部件极其复杂
- **格式少，且源寄存器位置相同，有利于在指令未知时就可取操作数**
 - MIPS指令的rs和rt位置一定，在指令译码时就可读rs和rt的值
 - 若位置随指令不同而不同，则需先确定指令类型才能取寄存器编号



流水线指令集的设计

- **load / Store指令才能访问存储器，有利于减少操作步骤，规整流水线**
 - lw/sw指令的地址计算和运算指令的执行步骤规整在同一个周期
 - X86运算类指令操作数可为内存数据，需计算地址、访存、执行
- **内存中"对齐"存放，有利于减少访存次数和流水线的规整**
- **规整、简单和一致等特性有利于指令的流水线执行**



【2009统考真题】某计算机的指令流水线由4个功能段组成，指令流经各功能段的时间(忽略各功能段之间的缓存时间)分别为90ns、80ns、70ns 和60ns，则该计算机的 CPU 周期至少是()。

A.90ns

B.80ns

C.70ns

D.60ns

■ A

【2018统考真题】若某计算机最复杂指令的执行需要完成5个子功能，分别由功能部件A~E实现，各功能部件所需时间分别为80ps、50ps、50ps、70ps和50ps，采用流水线方式执行指令，流水段寄存器延时为20ps，则CPU时钟周期至少为()。

A. 60ps

B. 70ps

C. 80ps

D. 100ps

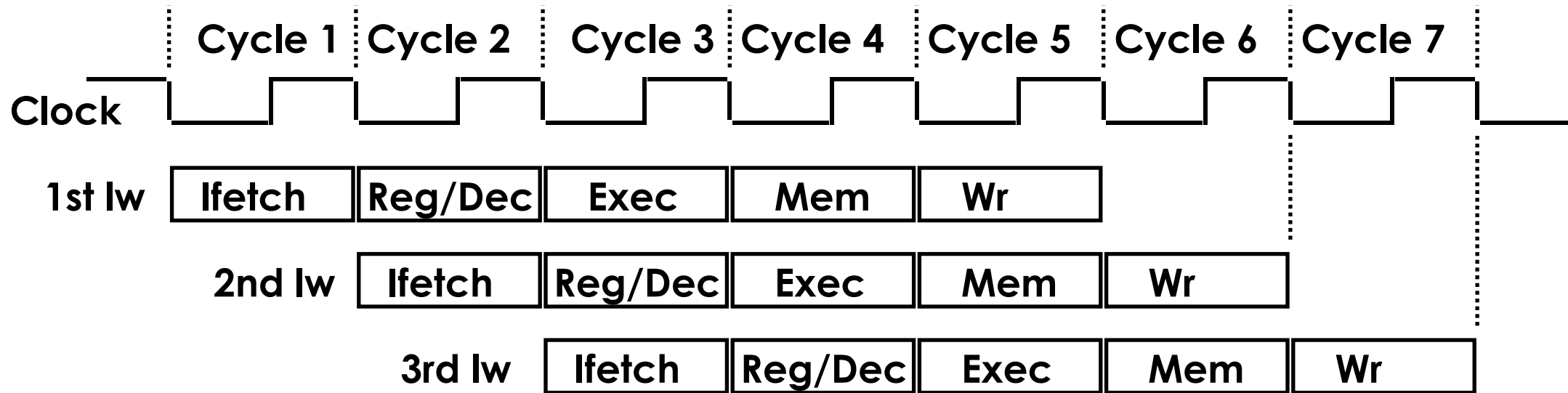
■ D

本章主要内容

- 6.1 流水线概述
- 6.2 流水线处理器的实现
- 6.3 流水线冒险及其处理

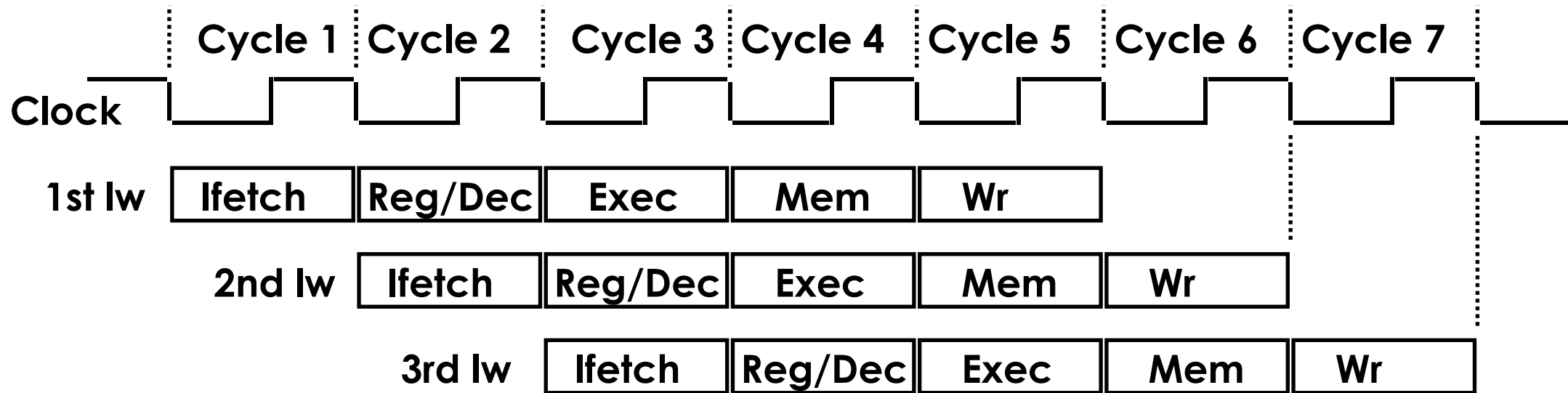


Load指令的流水线



- 每个周期有五个功能部件同时在工作
- 后面指令在前面完成取指后马上开始
- 每个load指令仍然需要五个周期完成

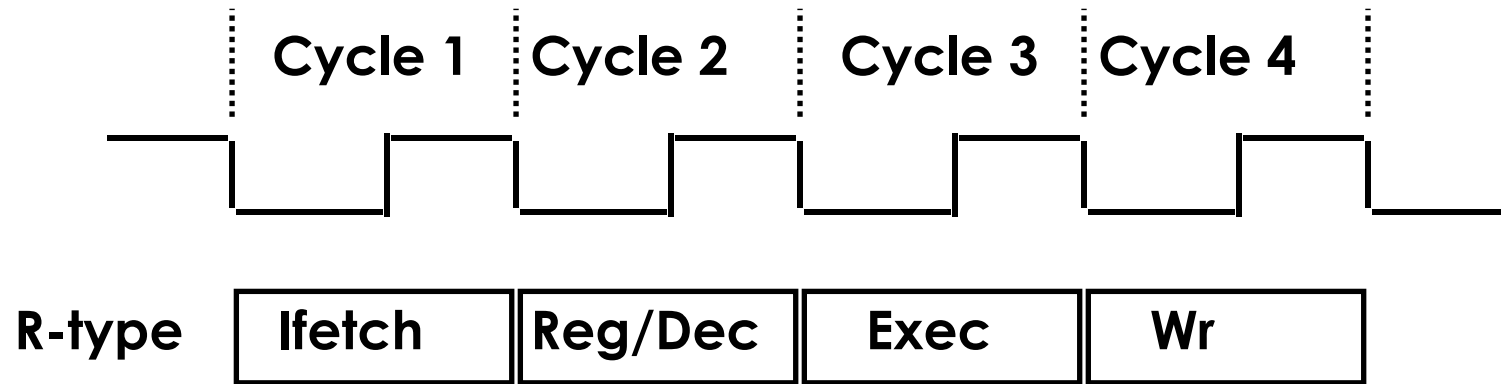
Load指令的流水线



单位时间内处理器执行指令的条数

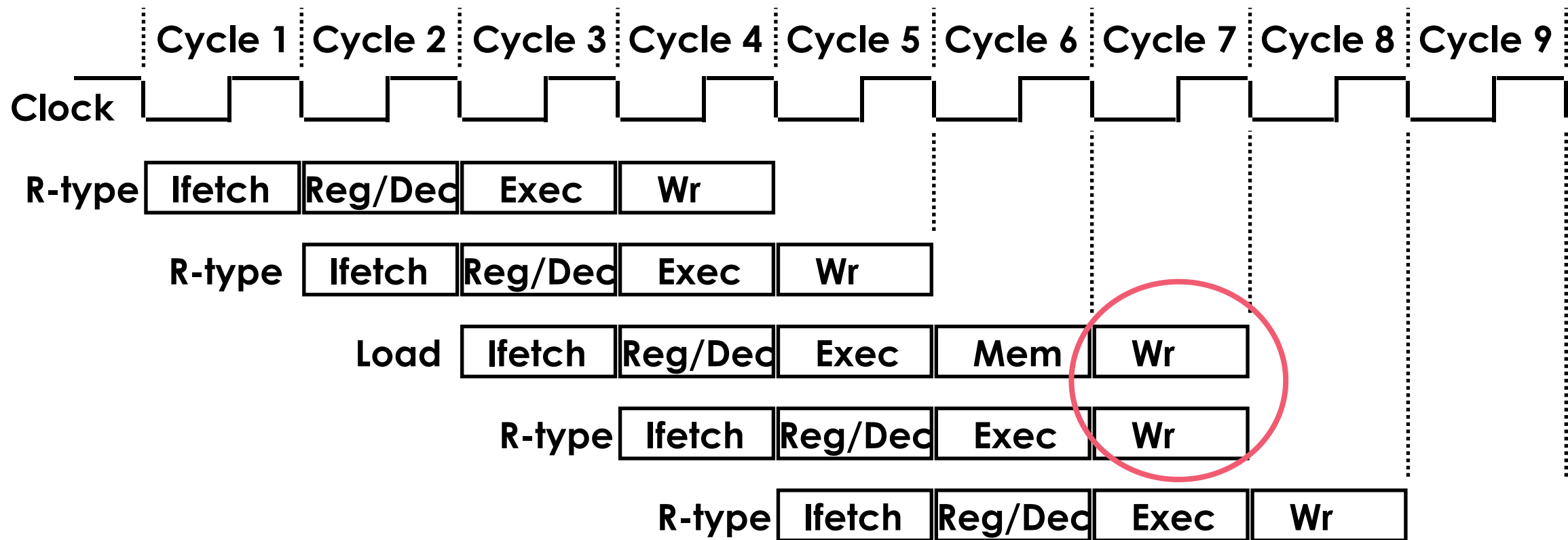
- 吞吐率提高许多，理想情况下，有：
 - 每个周期有一条指令进入流水线
 - 每个周期都有一条指令完成
 - 每条指令的有效周期(CPI)为1

R-type指令的4个阶段



- **Ifetch:** 取指令并计算PC+4
- **Reg/Dec:** 从寄存器取数，同时指令在译码器进行译码
- **Exec:** 在ALU中对操作数进行计算
- **Wr:** ALU计算的结果写到寄存器

含R-type和 Load 指令的流水线



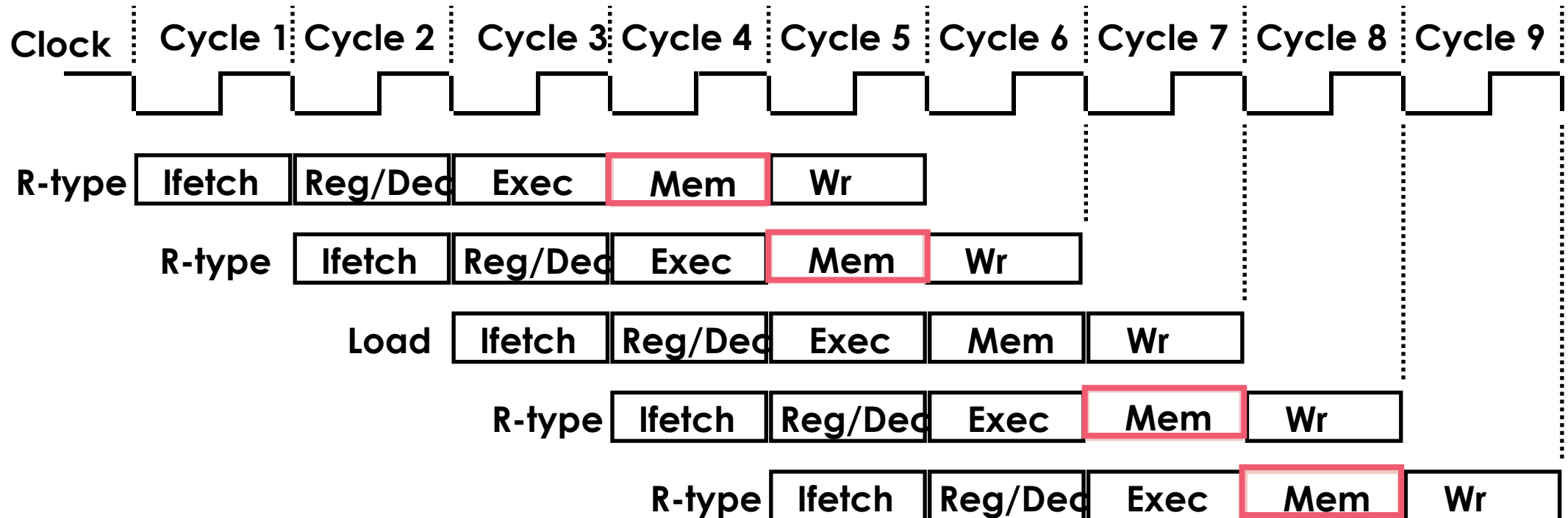
含R-type和 Load 指令的流水线

- 上述流水线有个问题：两条指令试图同时写寄存器，因为
 - Load在第5阶段用寄存器写口
 - R-type在第4阶段用寄存器写口
- 把一个功能部件同时被多条指令使用的现象称为**结构冒险**
- 为了流水线能顺利工作，规定：
 - 每个功能部件每条指令只能用一次（如：写口不能用两次或以上）
 - 每个功能部件必须在相同的阶段被使用（如：写口总是在第五阶段被使用）

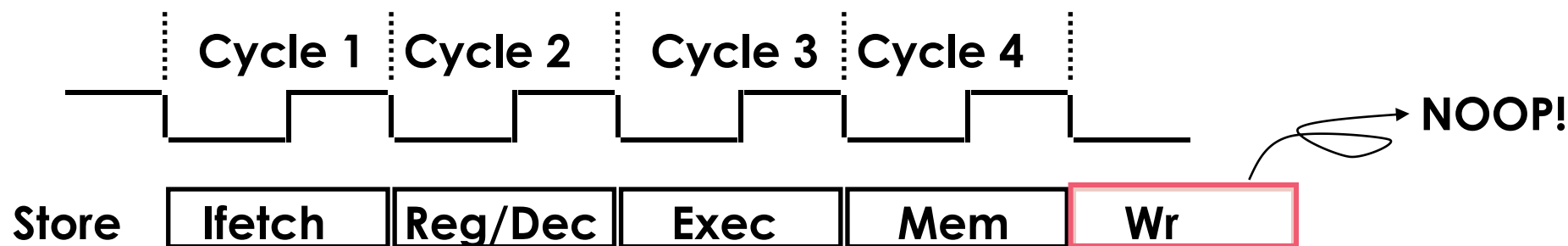
R-type的Wr操作延后一个周期执行



- 加一个NOP阶段以延迟"写"操作
- 把"写"操作安排在第5阶段, 这样使R-Type的Mem阶段为空NOP

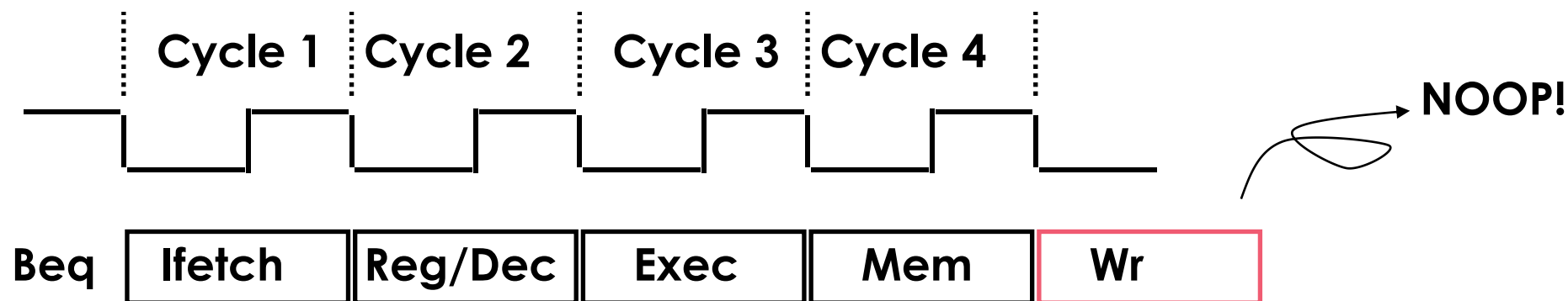


Store指令的四个阶段

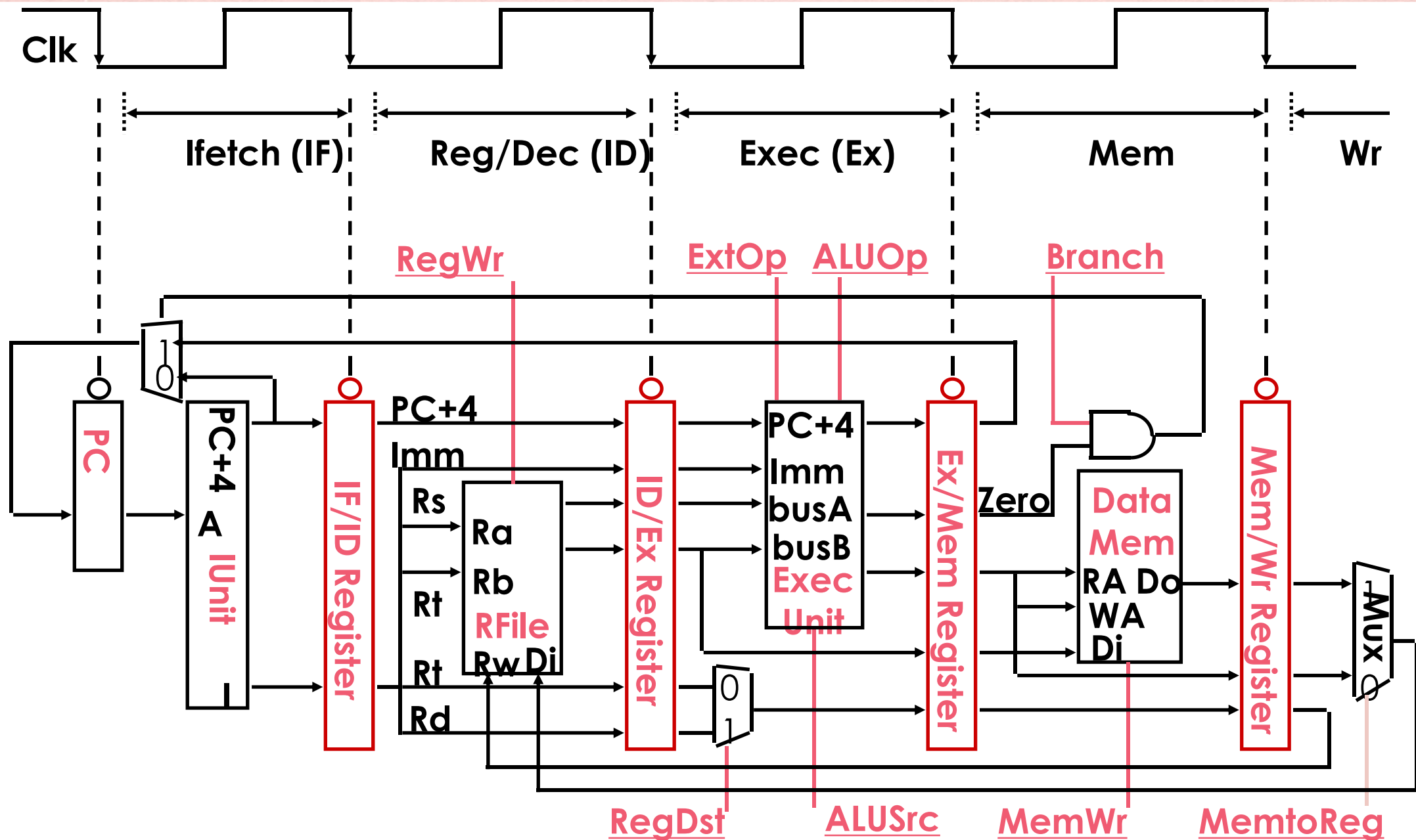


- Ifetch: 取指令并计算PC+4
- Reg/Dec: 从寄存器取数，同时指令在译码器进行译码
- Exec: 16位立即数符号扩展后与寄存器值相加，计算主存地址
- Mem: 将寄存器读出的数据写到主存
- Wr: 加一个空的写阶段，使流水线更规整

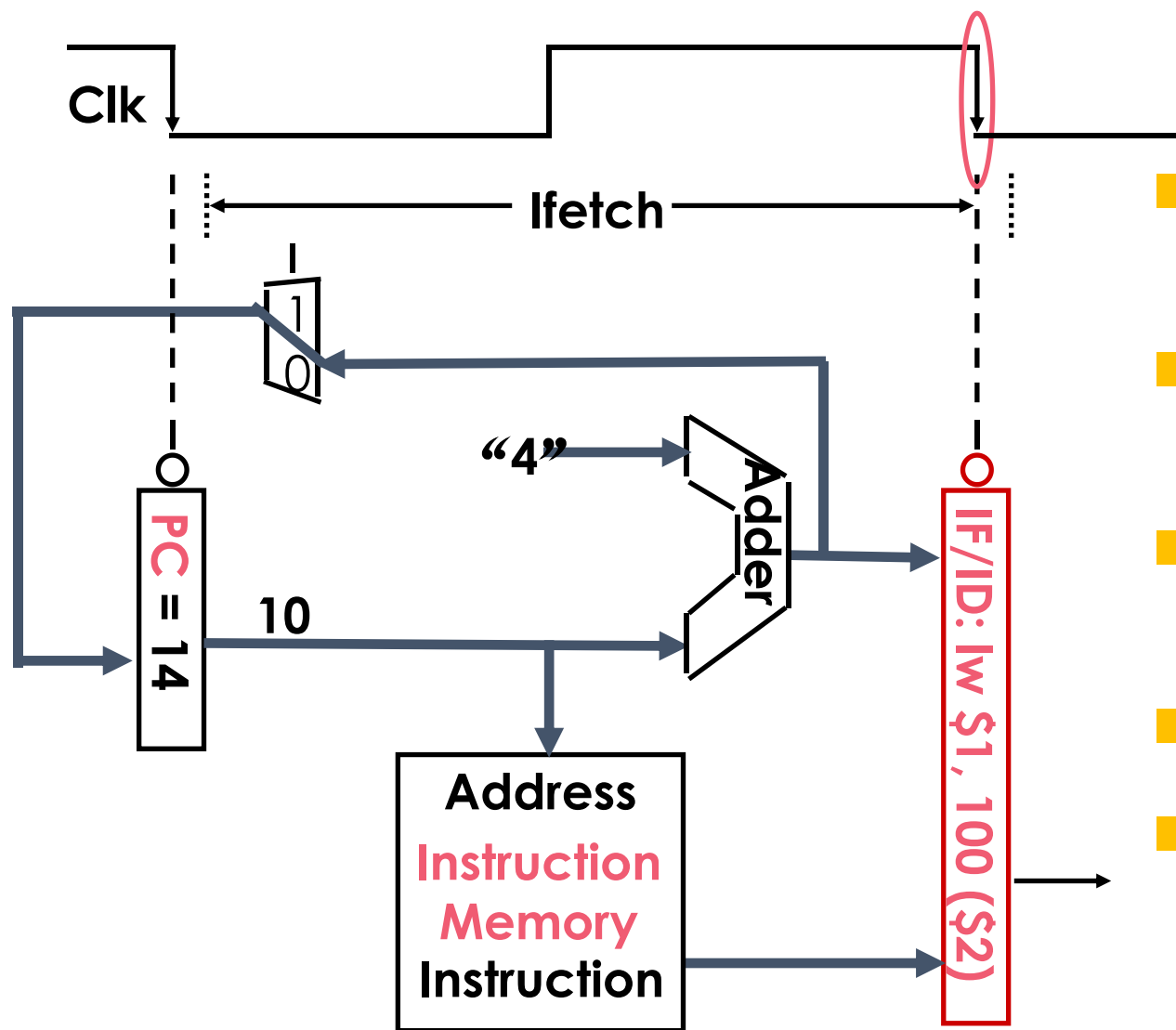
Beq的四个阶段



- Ifetch: 取指令并计算PC+4
- Reg/Dec: 从寄存器取数，同时指令在译码器进行译码
- Exec: 执行阶段
 - ——ALU中比较两个寄存器的大小（做减法）
 - ——Adder中计算转移地址
- Mem: 如果比较相等, 则：转移目标地址写到PC
- Wr: 加一个空写阶段，使流水线更规整



取指令(Ifetch)阶段指令部件Unit的设计

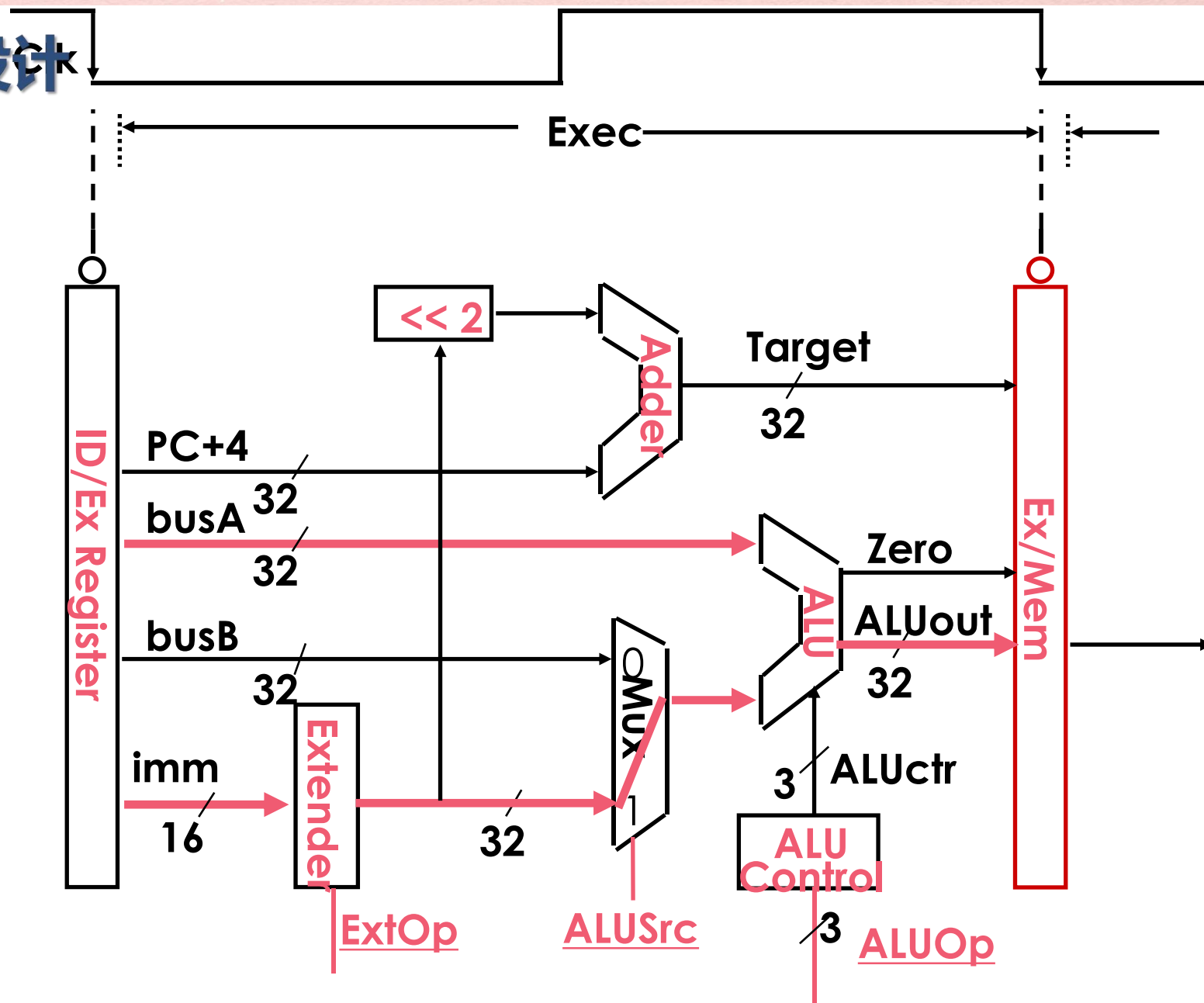


- **流水段寄存器**用来存放各阶段的执行结果
- 总是在下个时钟到来后的Clock-to-Q输出
- 应保存后面阶段用到的指令和PC+4的值
- 指令在随后阶段被送出译码
- PC+4用来计算转移目标地址

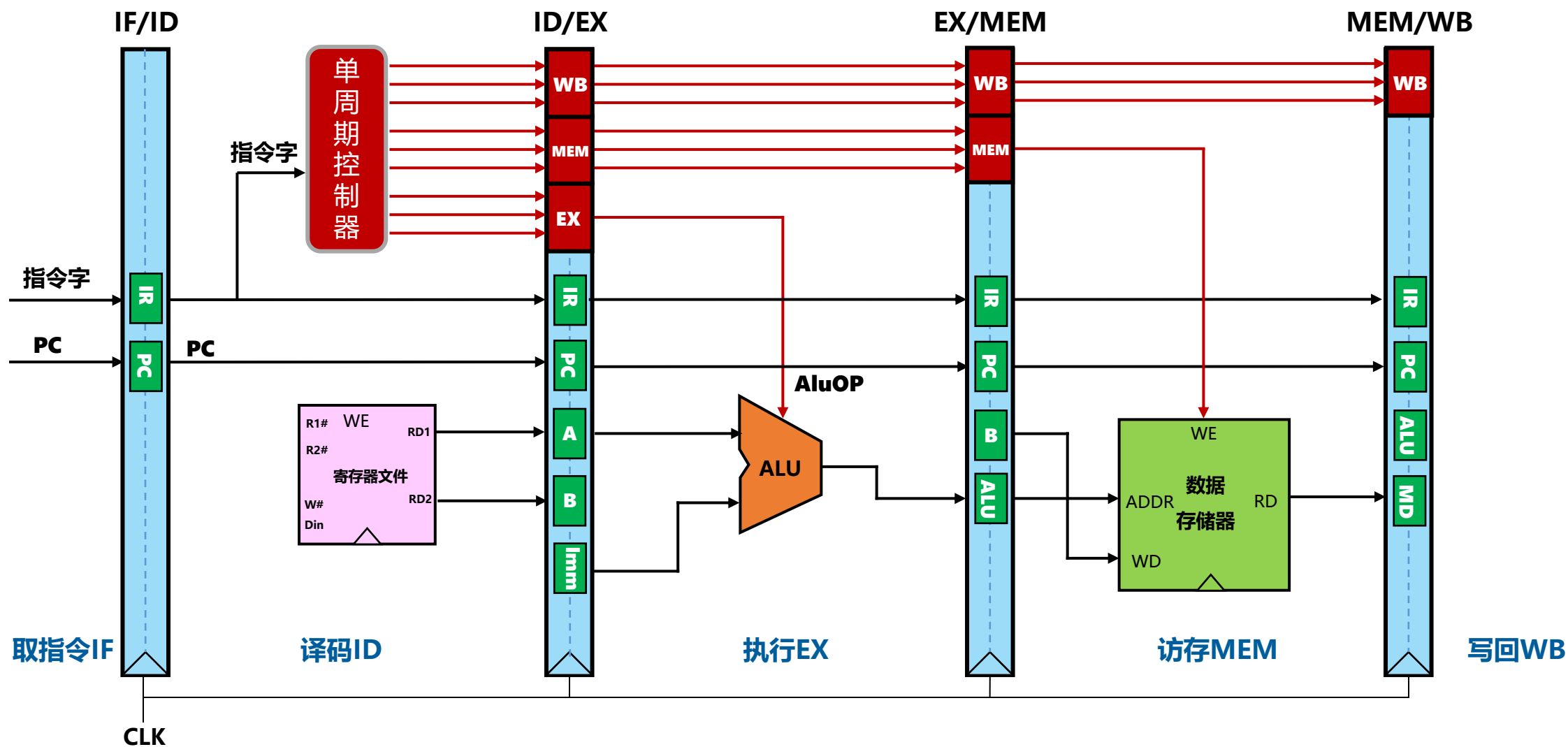
执行部件(Exec Unit)的设计

■ 执行部件功能

- 计算内存地址
- 计算转移目标地址
- 一般ALU运算



5段指令流水线数据与信号传递



■ 译码段生成各段所需的控制信号，依次向后传递

流水线中的控制信号

- **PC需要写使能吗？** 每个时钟都会改变PC，故不需要
- **流水段寄存器需要写使能吗？** 每个时钟都会改变流水段寄存器，故不需要
- **Ifecth阶段和Dec/Reg阶段都没有控制信号**
- **Exec阶段的控制信号有四个：**
 - ExtOp (扩展器操作)：1- 符号扩展；0- 零扩展
 - ALUSrc (ALU的B口来源)：1- 来源于扩展器；0- 来源于BusB
 - ALUOp (主控制器输出，用于辅助局部ALU控制逻辑来决定ALUCtrl)
 - RegDst (指定目的寄存器)：1- Rd；0- Rt

流水线中的控制信号

■ Mem阶段的控制信号有两个：

- MemWr (DM的写信号)：Store指令时为1，其他指令为0
- Branch (是否为分支指令)：分支指令时为1，其他指令为0

■ Wr阶段的控制信号有两个：

- MemtoReg (寄存器的写入源)：1- DM输出；0- ALU输出
- RegWr (寄存器堆写信号)：结果写寄存器的指令都为1，其他指令为0

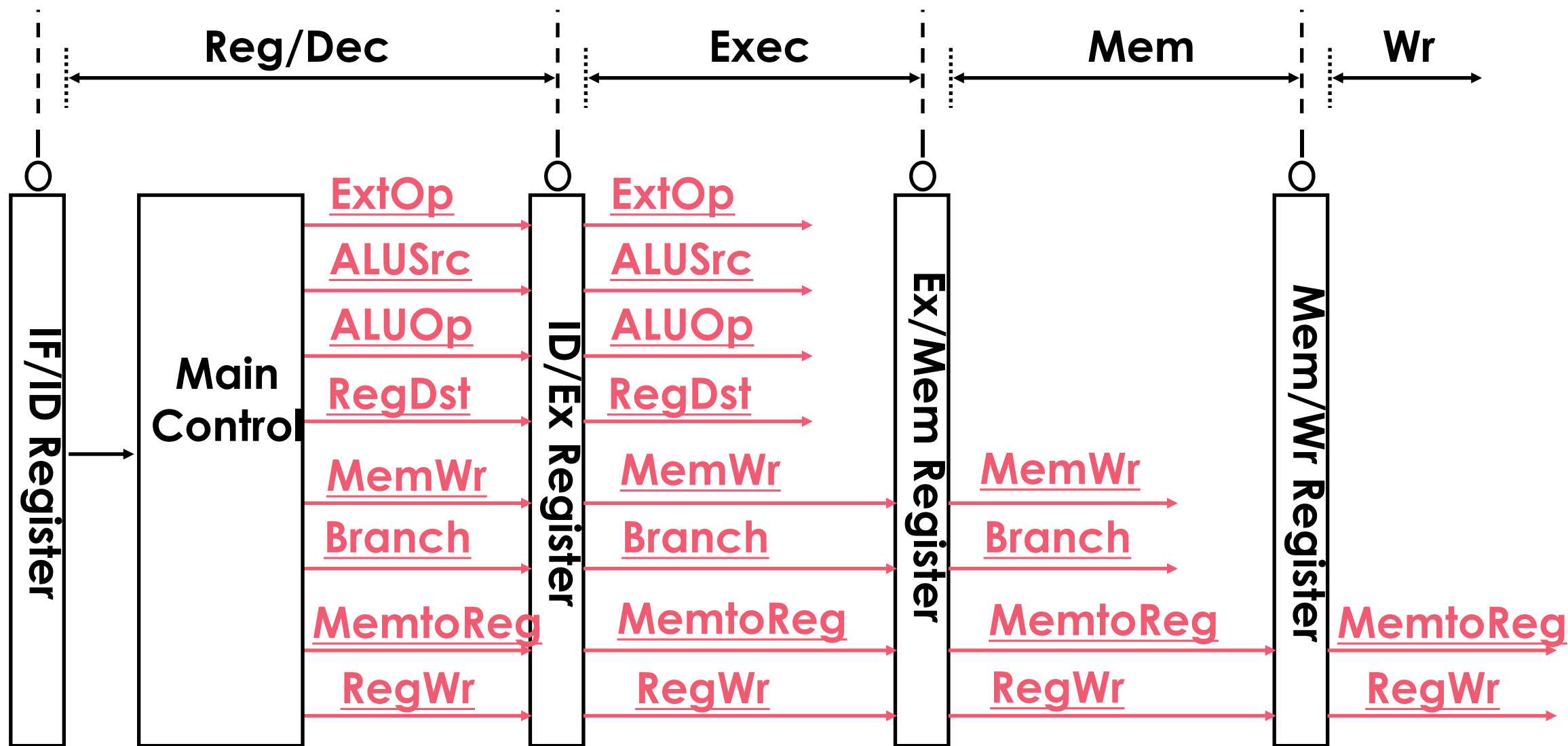
■ 控制逻辑的输出（控制信号）在ID阶段生成，并存放在ID/EX流水段寄存器中，然后每来一个时钟跟着指令传送到下一级流水段寄存器

■ 某时刻在不同阶段同时执行不同指令，不同的指令得到不同控制信号

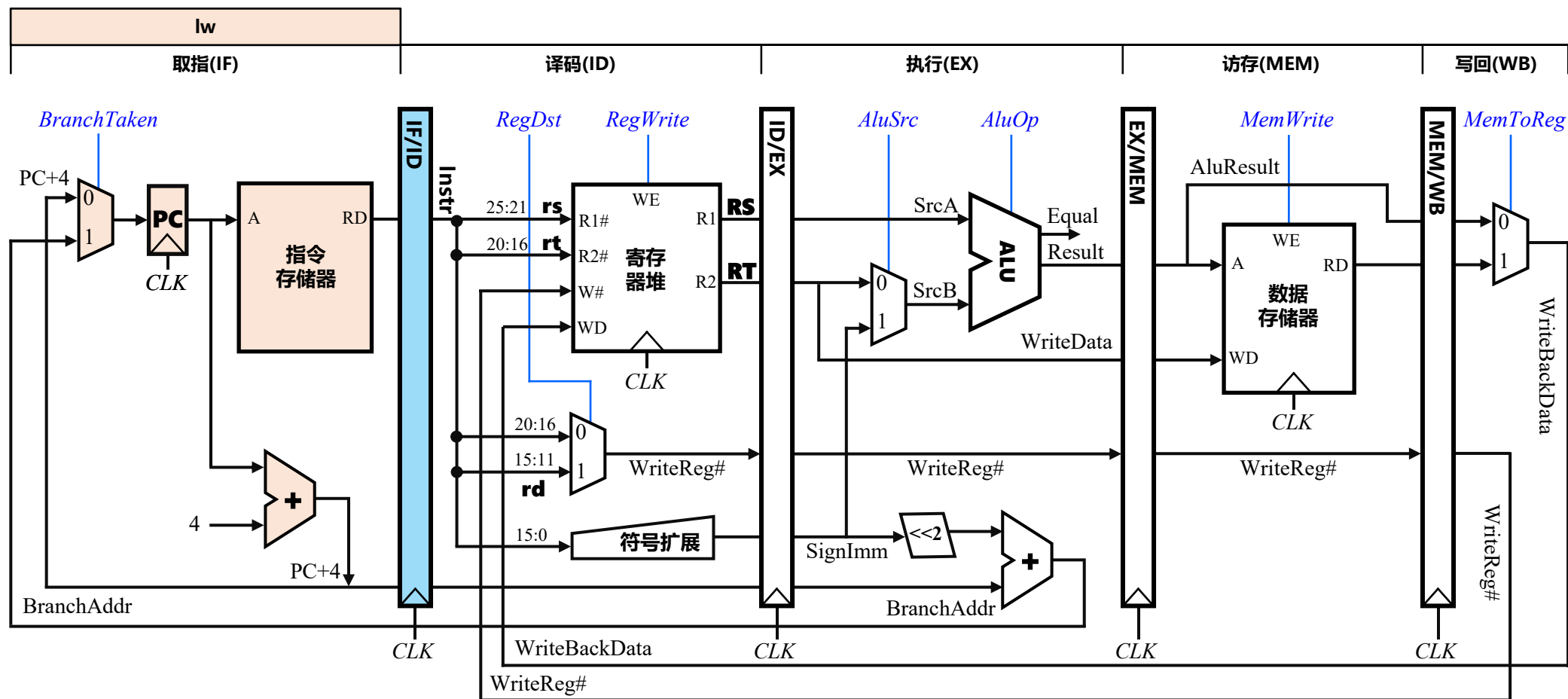
控制信号分类

控制信号	位置	来源	功能说明
BranchTaken	IF	EX	分支跳转信号，为 1 表示跳转，由 EX 段的 Branch 信号与 equal 标志进行逻辑与生成
RegDst	ID	ID	写入目的寄存器选择，为 1 时目的寄存器为 rd 寄存器，为 0 时为 rt 寄存器
RegWrite	ID	WB	控制寄存器堆写操作，为 1 时数据需要写回寄存器堆中的指定寄存器
AluSrc	EX	EX	ALU 的第二输入选择控制，为 0 时输入寄存器 rt，为 1 时输入扩展后的立即数
AluOp	EX	EX	控制 ALU 进行不同运算，具体取值和位宽与 ALU 的设计有关
MemWrite	MEM	MEM	控制数据存储器写操作，为 0 时进行读操作，为 1 时进行写操作
MemToReg	WB	WB	为 1 时将数据存储器读出数据写回寄存器堆，否则将 ALU 运算结果写回

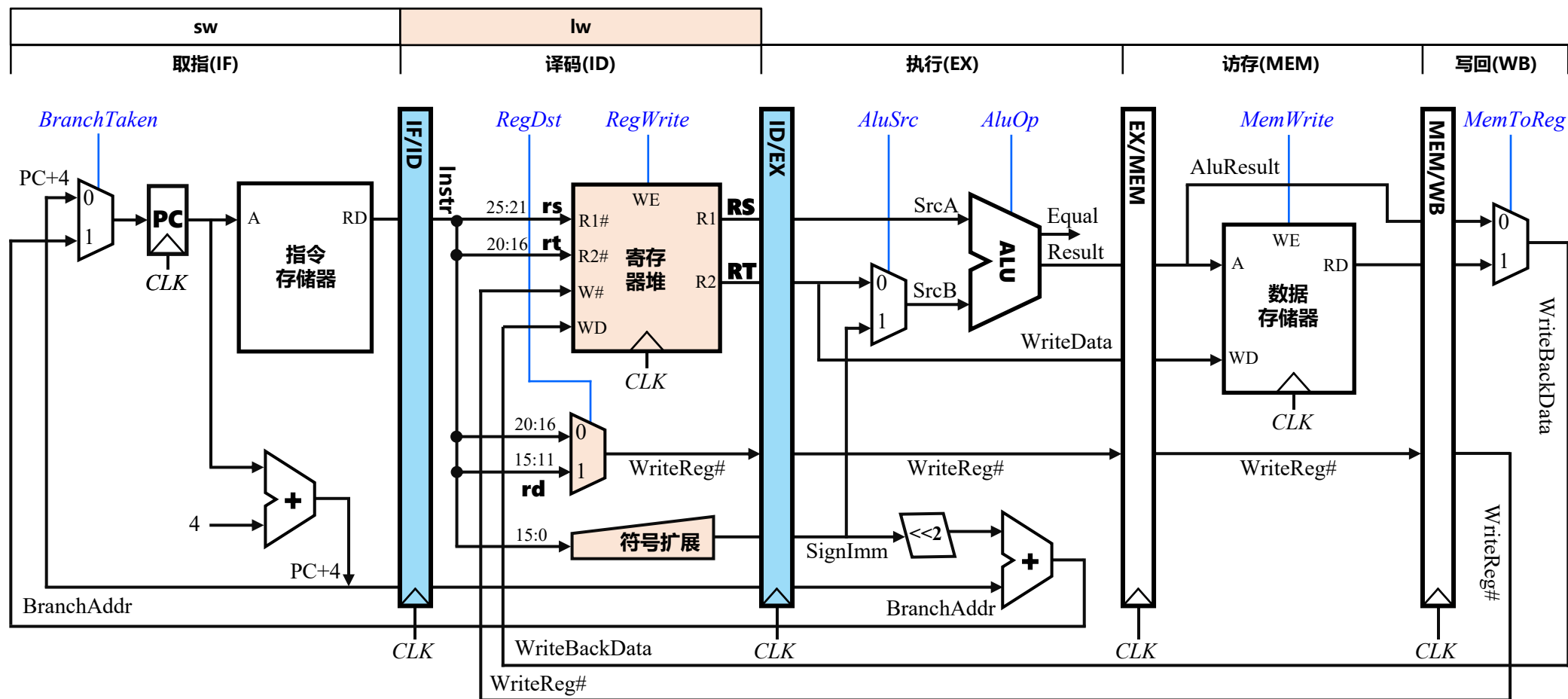
5段流水线控制信号与传递



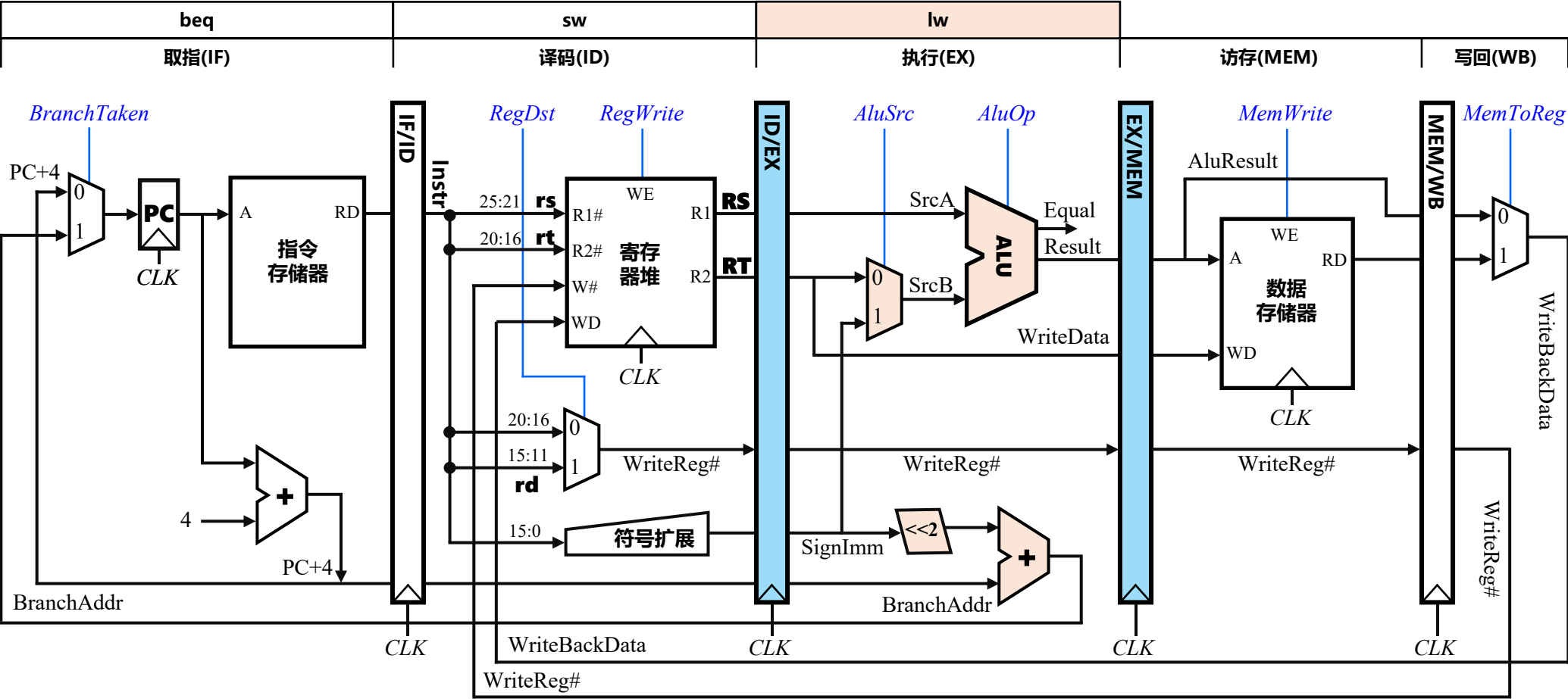
指令在流水线中的执行过程



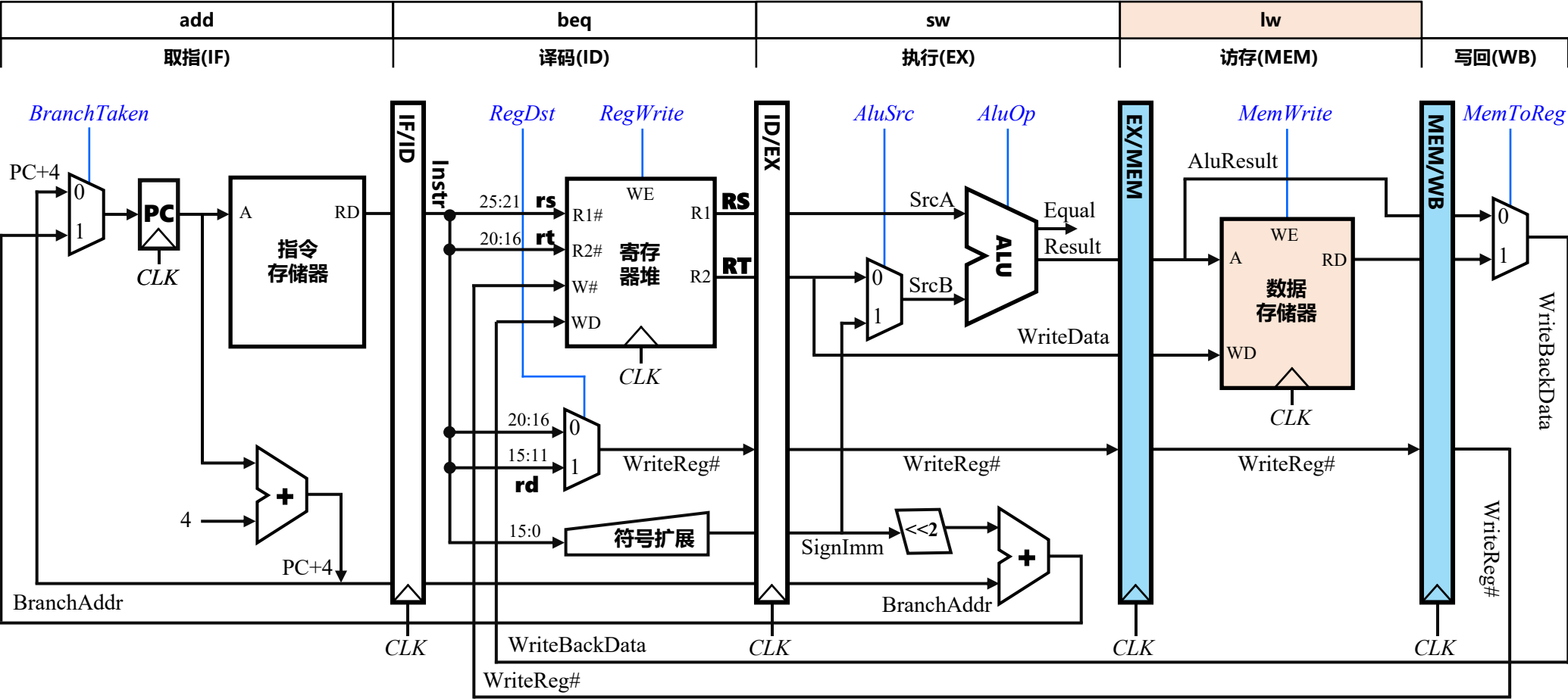
指令在流水线中的执行过程



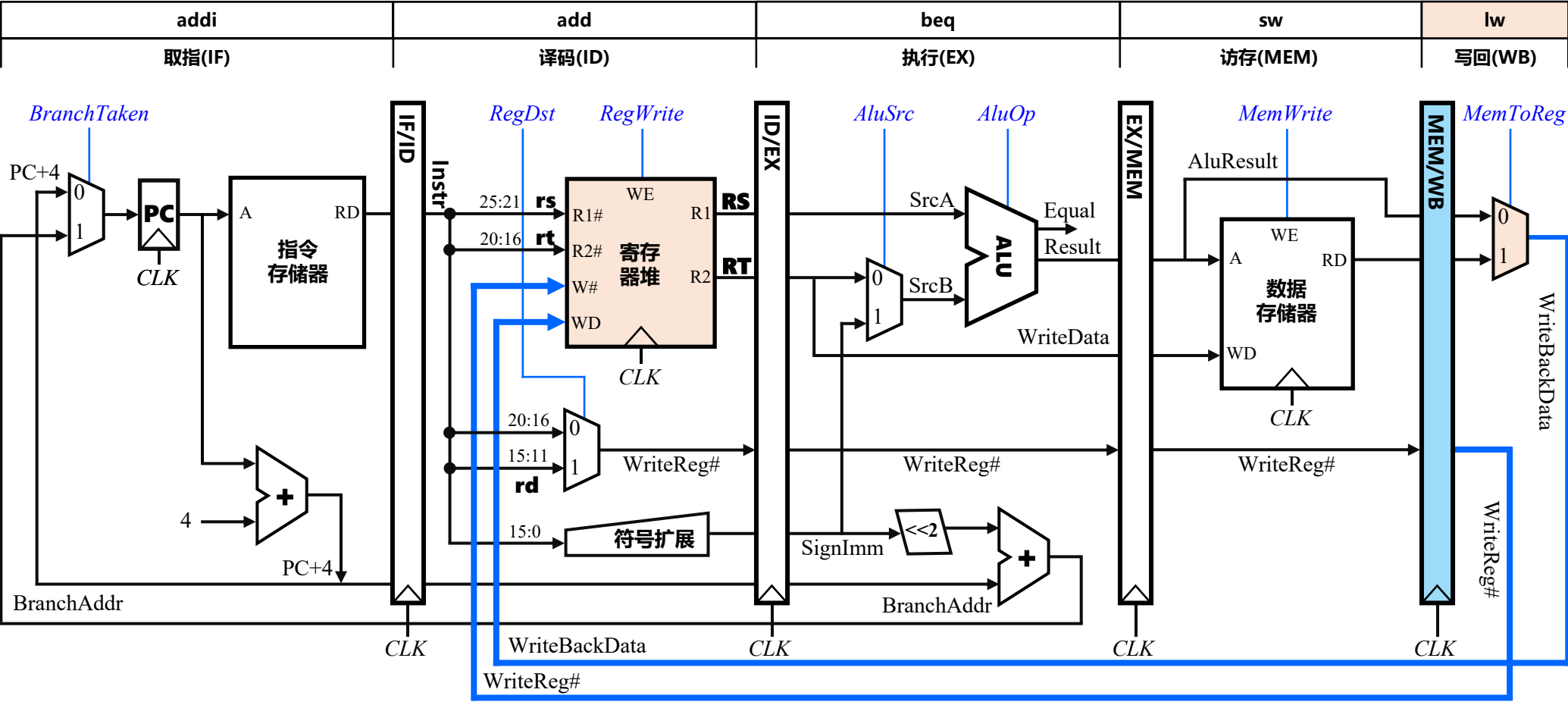
指令在流水线中的执行过程



指令在流水线中的执行过程



指令在流水线中的执行过程



本章主要内容

- 6.1 流水线概述
- 6.2 流水线处理器的实现
- 6.3 流水线冒险及其处理



指令流水线的冲突、冒险 (hazard)

- 冒险：指流水线遇到无法正确执行后续指令或执行了不该执行的指令
- 结构冒险
 - 现象：同一个部件同时被不同指令所使用
- 数据冒险
 - 现象：后面指令用到前面指令结果数据时，前面指令的结果还没产生
- 控制冒险
 - 现象：转移或异常改变执行流程，后继指令在目标地址产生前已被取出

指令流水线的冲突、冒险 (hazard)

■ 结构冲突

- **争用主存**: IF段取指令、ID段取操作数
- **争用ALU**: 多周期方案中计算PC、分支地址, 运算指令
- **解决方案**: 增加部件消除

■ 数据冲突

- 指令操作数依赖于前一条指令的执行结果
- 引起流水线停顿直到数据写回

ADD **\$s1**, \$s2, \$s3

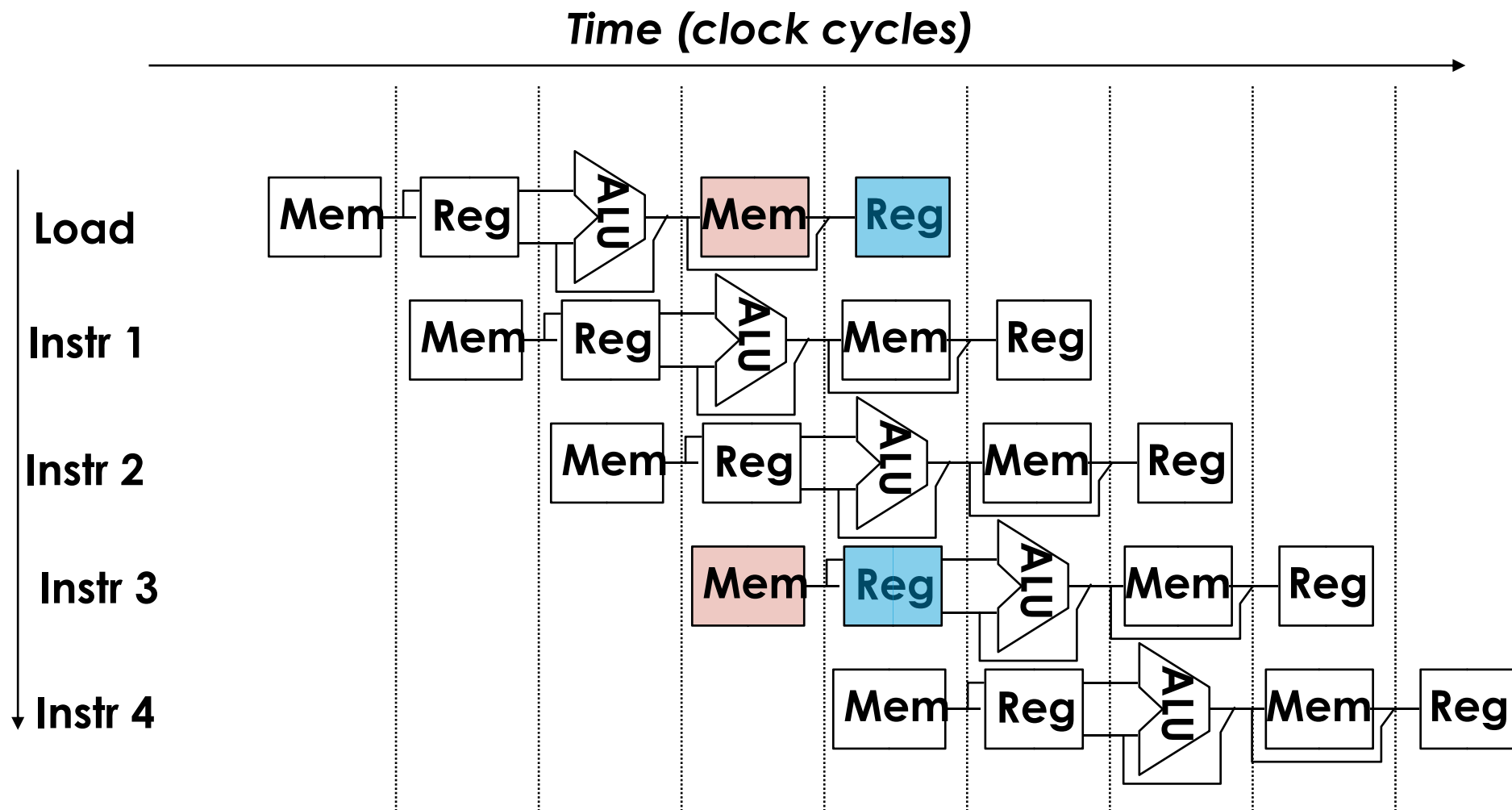
ADD \$s4, **\$s1**, \$s3

■ 控制冲突

- 提前取出的指令作废, 流水线清空
- 流水线发生中断

结构冒险

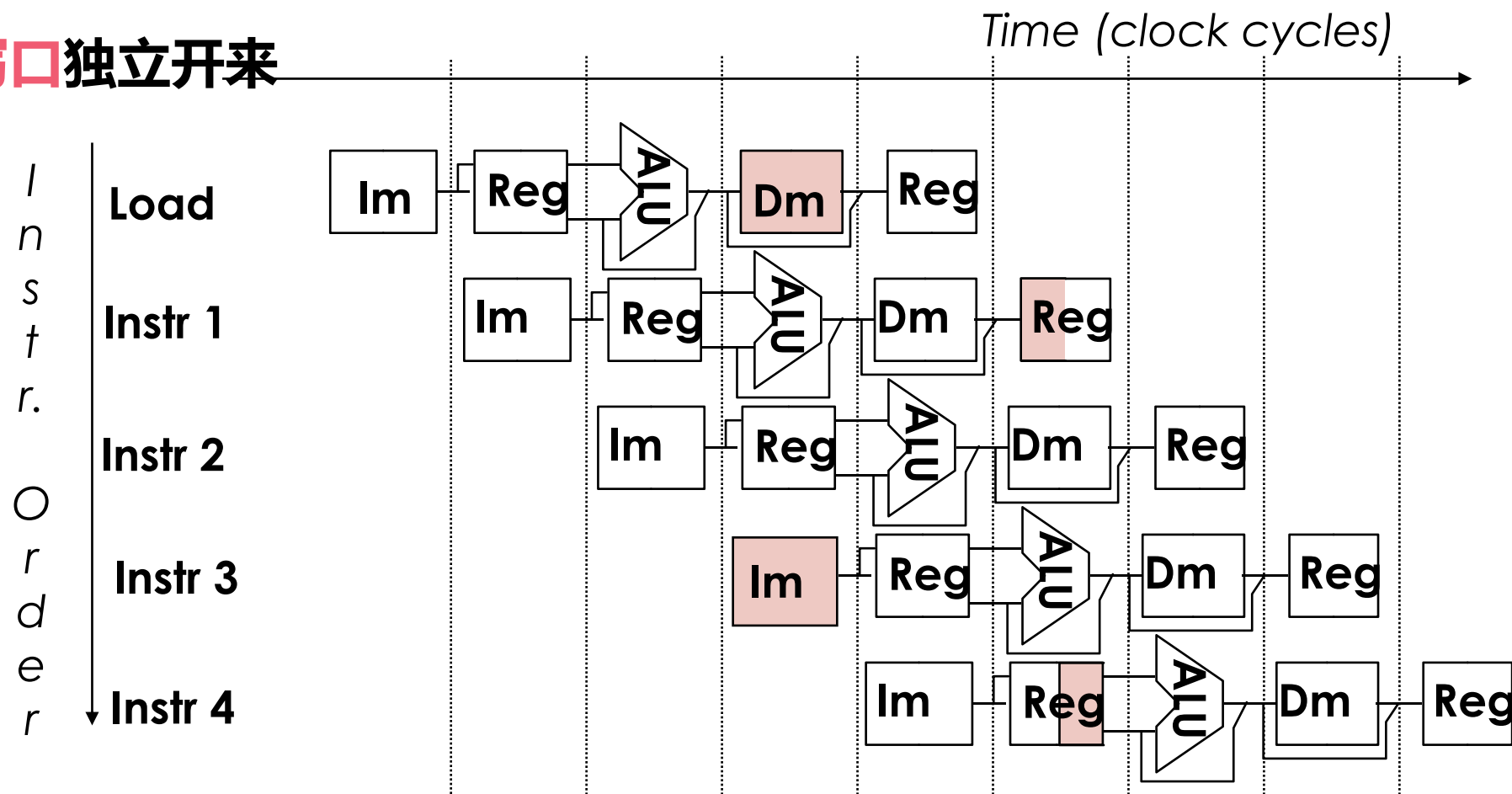
结构冒险也称硬件资源冲突：同一个执行部件被多条指令使用。



- 只有一个存储器时，在Load指令取数据同时又取指令的话，则发生冲突！
- 如果不对寄存器堆的写口和读口独立设置的话，则发生冲突！

为了避免结构冒险，规定流水线数据通路中功能部件的设置原则为：

- 一个部件每条指令只能使用1次，且只能在特定周期使用
- 将指令存储器(Im)和数据存储器(Dm)分开
- 将寄存器读口和写口独立开来



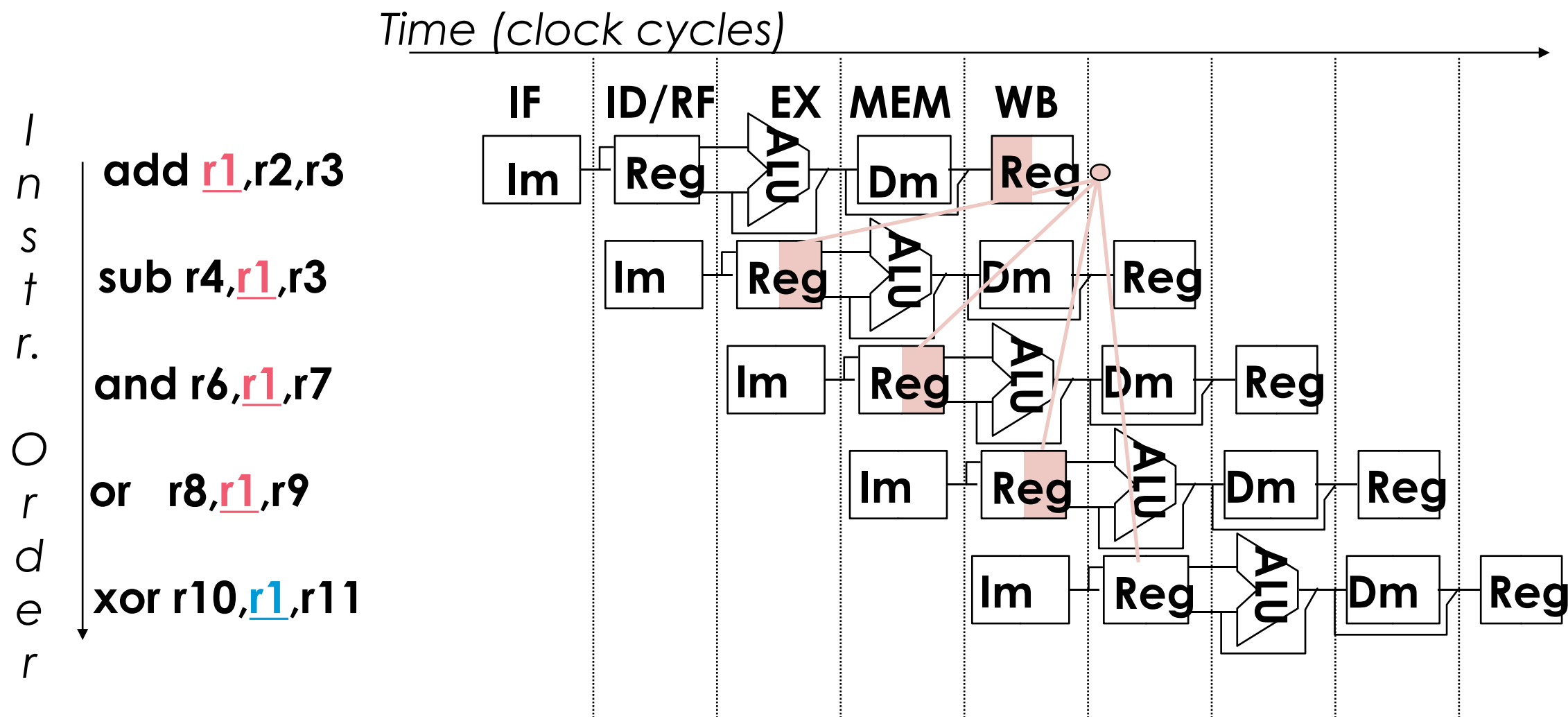
数据冒险

- 现象：在指令流水线中后面指令用到前面指令结果数据时，前面指令的结果还没产生。

□ add r1, r2, r3	读r1时，add指令正在执行加法(EXE)，老值
□ sub r4, r1, r3	读r1时，add指令正在传递加法结果(MEM)，老值
□ and r6, r1, r7	读r1时，add指令正在写加法结果到r1(WB)，老值
□ or r8, r1, r9	读r1时，add指令已经把加法结果写到r1，新值
□ xor r10, r1, r11	

- 想一下，哪条指令的r1是老的值？哪条是新的值？
- 由于前面指令写结束之前后面指令就需要读取而造成的冒险称为写后读（RAW）数据冒险。

数据冒险



数据冒险的解决方法

◆ 软件方法（编译器完成）

◆ 插入空操作指令

◆ 调整程序顺序，使相关性在流水线中消失

◆ 硬件方法

◆ 插入气泡（空操作）

- 向后段插入气泡（接口信号清零）

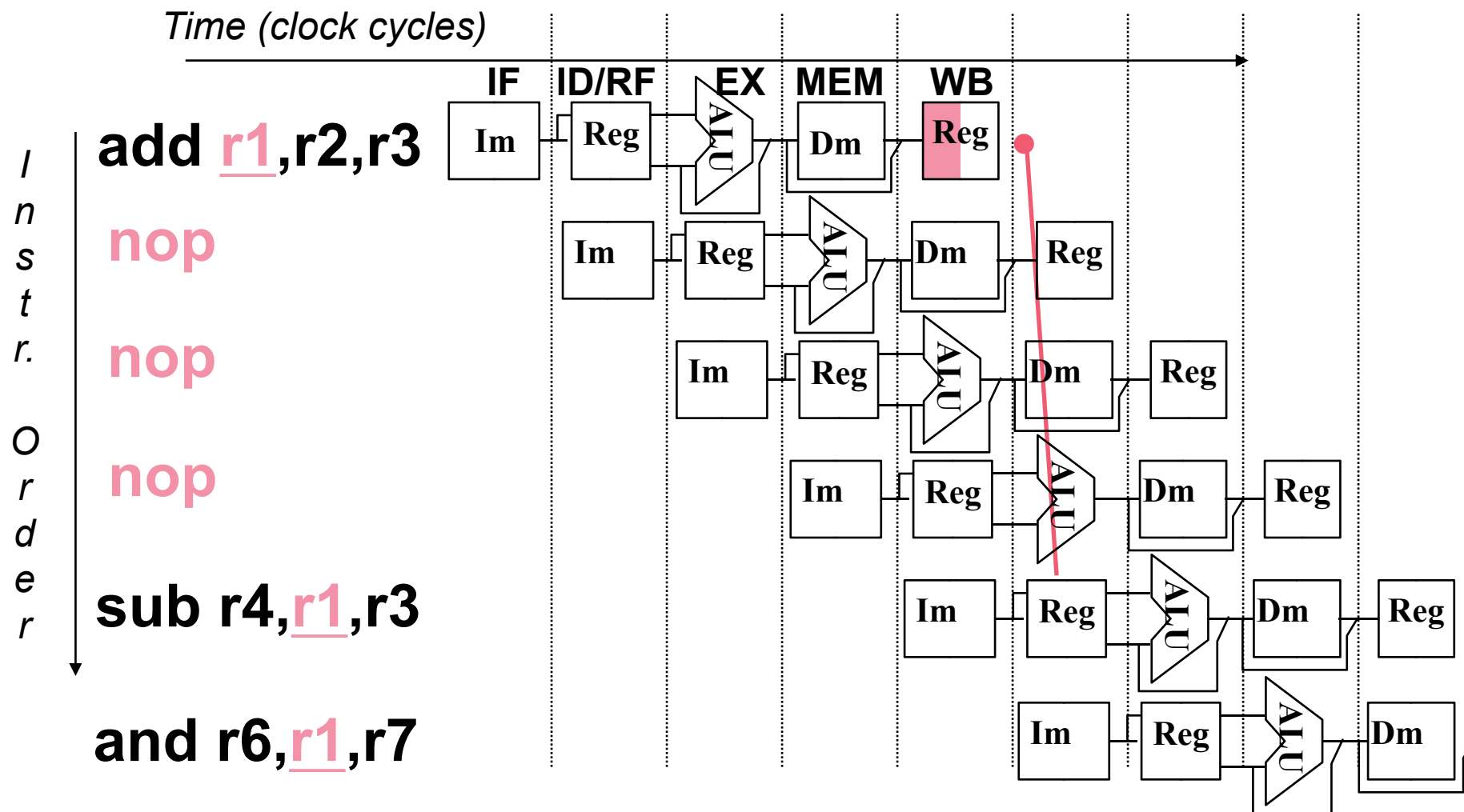
- 向前给出阻塞信号（流水线停顿）避免当前指令被新指令取代

◆ 数据重定向bypass（数据旁路）

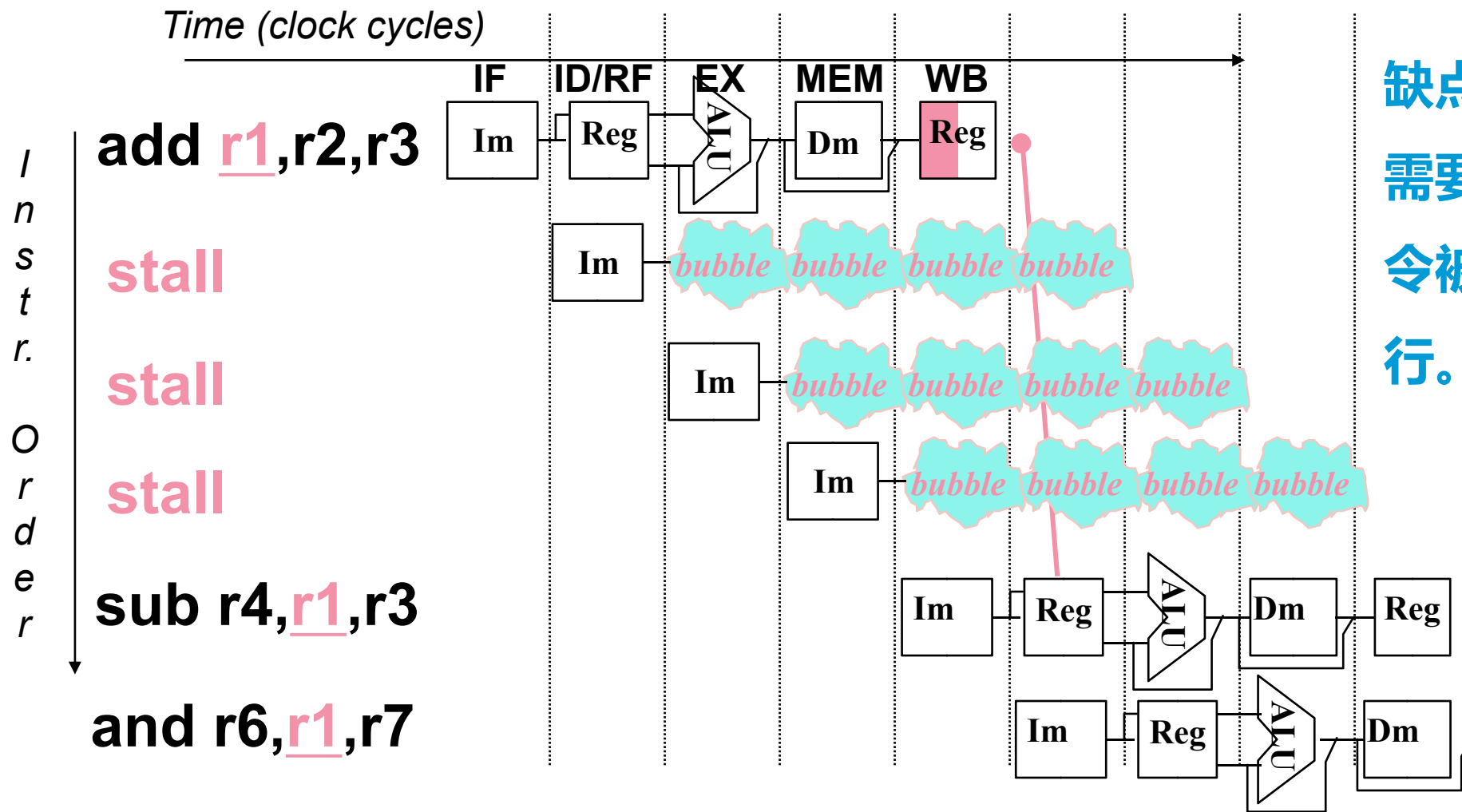
- ◆ 将后端处理后的数据（还没来得及写回）重定向

- ◆ 数据在哪就从哪送到运算器

- 由编译器插入三条NOP指令，浪费三条指令的空间和时间，是最差的做法。
- 好处：数据通路简单，即无需改数据通路。

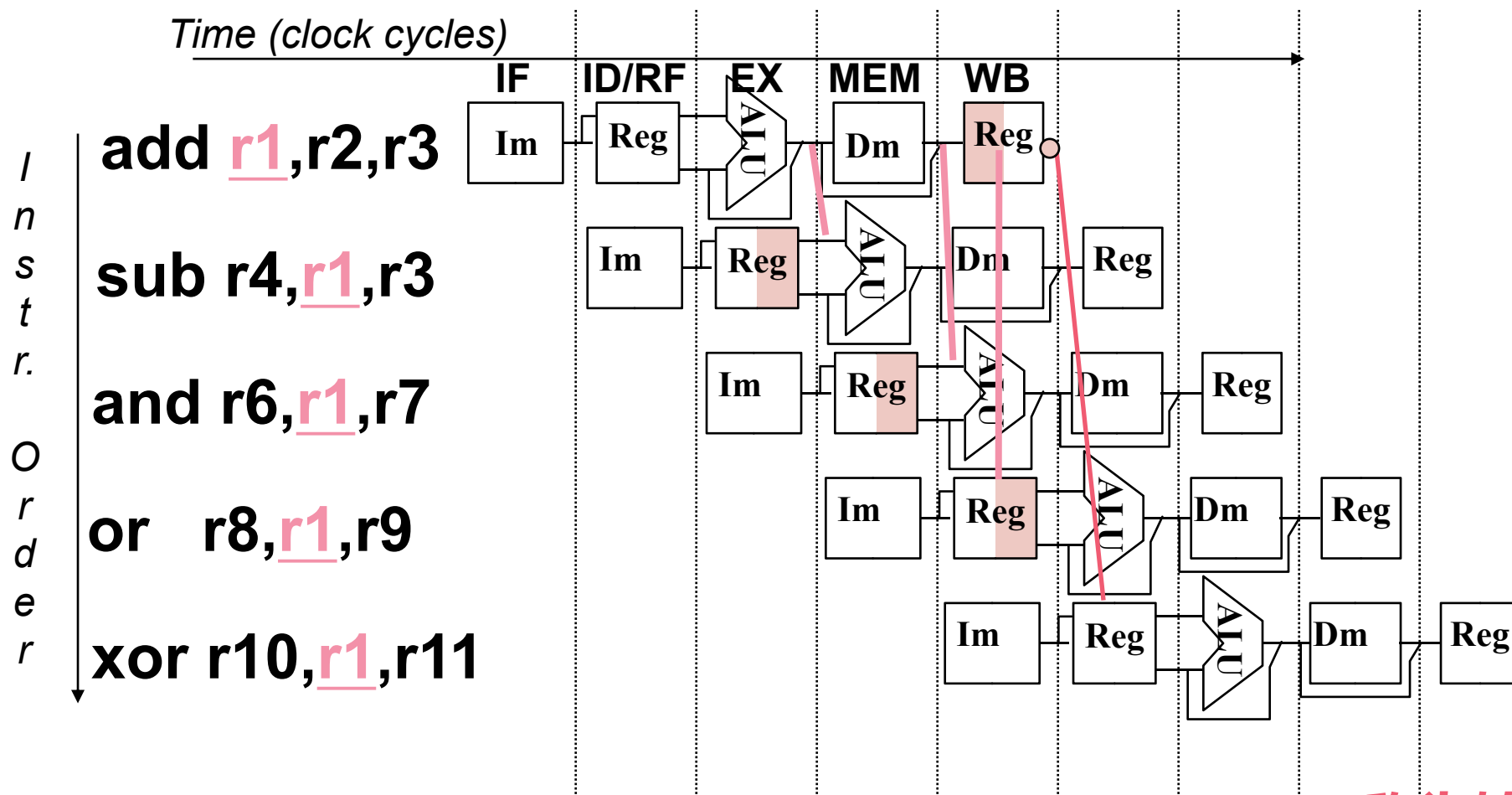


- 硬件上通过阻塞(stall)方式阻止后续指令执行，延迟到有新值以后！
- 这种做法称为流水线阻塞，也称为插入"气泡Bubble"



缺点：控制比较复杂，
需要改数据通路；指
令被延迟三个时钟执
行。

- 仔细观察后发现：流水段寄存器中已有需要的值r1！



- 把数据从流水段寄存器中直接取到ALU的输入端

称为转发(Forwarding)
或旁路(Bypassing)

【2016统考真题】在无转发机制的五段基本流水线(取指、译码/读寄存器、运算、访存、写回寄存器)中, 下列指令序列存在数据冒险的指令对是()。

I1 add R1, R2, R3; (R2)+(R3)→R1

I2 add R5, R2, R4; (R2)+(R4)→R5

I3 add R4, R5, R3; (R5)+(R3)→R4

I4 add R5, R2, R6; (R2)+(R6)→R5

A.I1和I2

B.I2和I3

C.I2和I4

D.I3和I4

■ B

【2019统考真题】在采用“取指、译码/取数、执行、访存、写回”5段流水线的处理器中，执行如下指令序列，其中s0、s1、s2、s3和t2表示寄存器编号。

I1: add s2,s1,s0 // $R[s2] \leftarrow R[s1] + R[s0]$

I2: load s3,0(t2) // $R[s3] \leftarrow M[R[t2] + 0]$

I3: add s2,s2,s3 // $R[s2] \leftarrow R[s2] + R[s3]$

I4: store s2,0(t2) // $M[R[t2] + 0] \leftarrow R[s2]$

下列指令对中，不存在数据冒险的是（ ）。

A. I1 和 I3

B. I2 和 I3

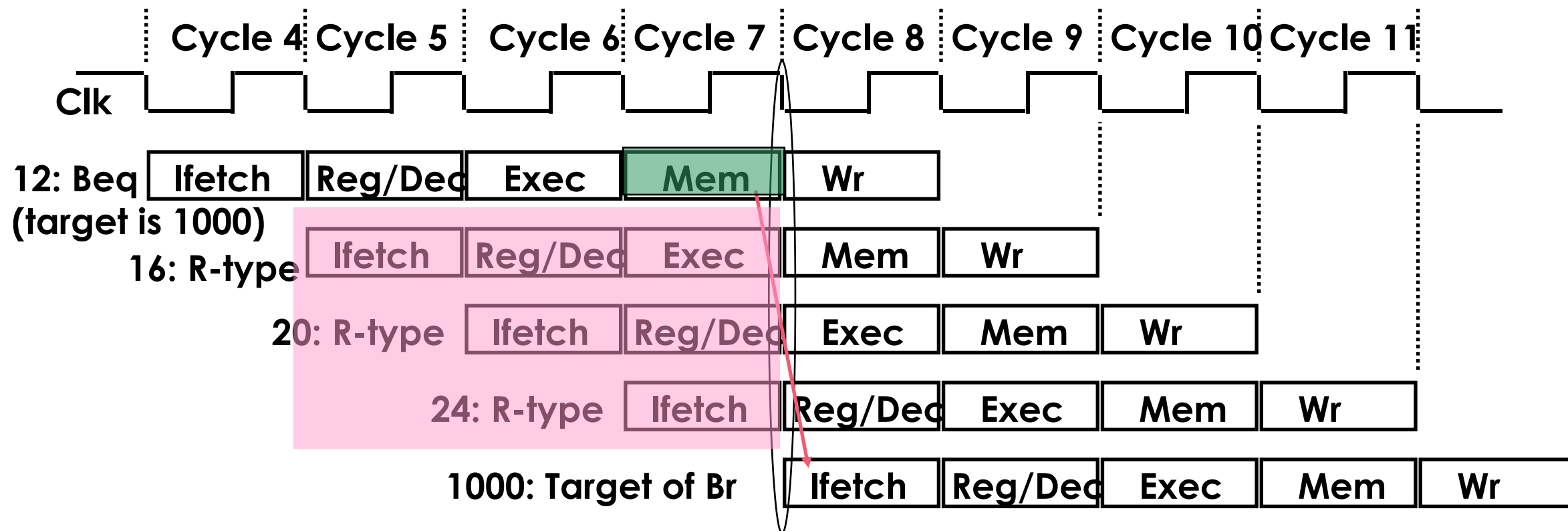
C. I2 和 I4

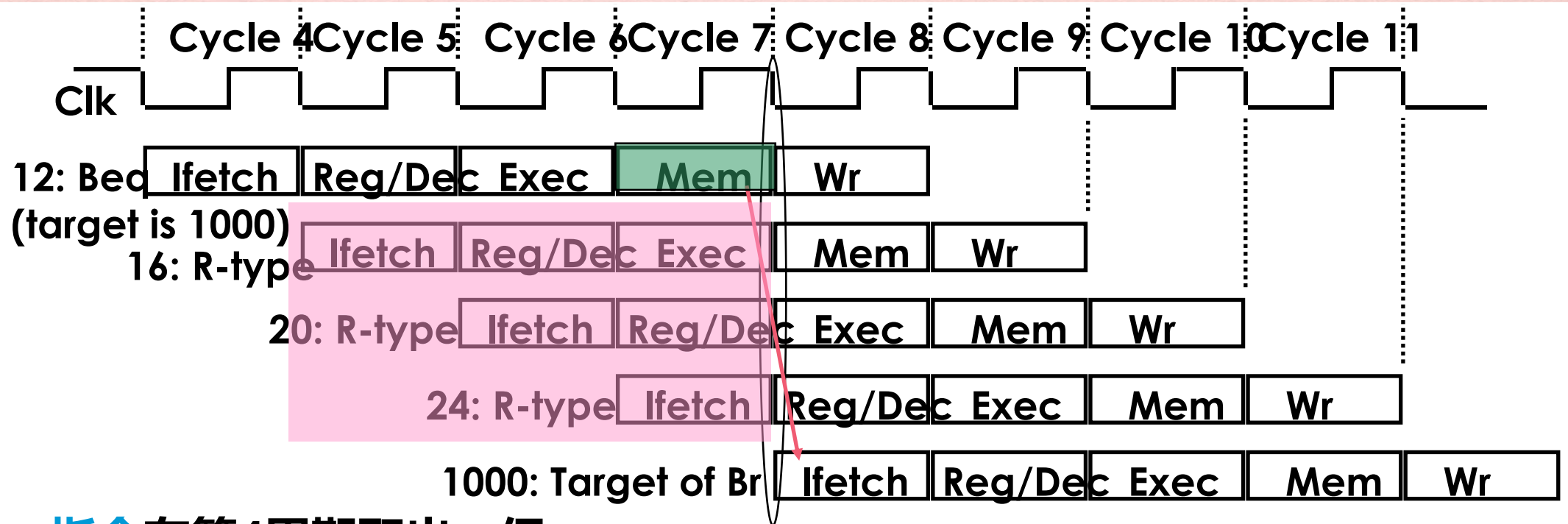
D. I3 和 I4

■ C

控制冒险

现象：转移或异常改变执行流程，后继指令在目标地址产生前已被取出





- 虽然Beq指令在第4周期取出，但：
 - “是否转移” 在Mem阶段确定，目标地址在第7周期才被送到PC输入端
 - 第8周期才取出目标地址处的指令执行
- 结果：在取目标指令之前，已有三条指令被取出，取错了三条指令！
- 发生转移时，要在流水线中清除Beq后面的三条指令，分别在EXE、ID、IF段中
- 延迟损失时间片C：由于流水线阻塞造成的延迟执行周期数。C = 3

控制冒险的解决方法

- ◆ 硬件阻塞方式（插入气泡）

- ◆ 软件阻塞方式（插入空操作指令）

- ◆ 分支预测（Predict）

 - ◆ 简单预测（静态预测）

 - ◆ 总是预测条件不满足或者条件满足

 - ◆ 可加启发式规则：在特定情况下总是预测满足(taken)，其他情况总是预测不满足。

 - ◆ 动态预测

 - ◆ 利用最近转移发生的情况，来预测下一次可能转移还是不转移，根据实际情况来调整预测

 - ◆ 转移发生的历史情况记录在一个表中

 - ◆ 每个表项由分支指令地址低位作索引，故在IF阶段就可以取到预测位

- 延迟分支

 - 通过编译程序优化指令顺序，把分支指令前面与分支指令无关的指令调到分支指令后执

【2010统考真题】 下列不会引起指令流水线阻塞的是()。

A.数据旁路

B.数据相关

C.条件转移

D.资源冲突

■ A

【2023统考真题】在采用"取指、译码/取数、执行、访存、写回"5段流水线的RISC处理器中, 执行如下指令序列(第一列为指令序号), 其中s0、s1、s2、s3和t2表示寄存器编号。

I1	add s2, s1, s0	// $R[s2] \leftarrow R[s1] + R[s0]$
I2	load s3, 0(s2)	// $R[s3] \leftarrow M[R[s2] + 0]$
I3	beq t2, s3, L1	//if $R[t2] = R[s3]$ jump to L1
I4	addi t2, t2, 20	// $R[t2] \leftarrow R[t2] + 20$
I5	L1:	

若采用转发(旁路)技术处理数据冒险, 采用硬件阻塞方式处理控制冒险, 则在指令I1~I4的执行过程中, 发生流水线阻塞的指令有()。

- A. 仅I3 B. 仅I2、I4 C. 仅I3、I4 D. 仅I2、I3、I4

■ C

【2024统考真题】对于采用"取指、译码/取数、执行、访存、写回"5段流水线的RISC数据通路，下列关于指令流水线**数据冒险处理**的叙述中，错误的是()。

- A. 相邻两条指令中的操作数相关可能引起数据冒险
- B. 在数据相关的指令间插入"气泡"能避免数据冒险
- C. 所有数据冒险都可以通过加入转发（旁路）电路解决
- D. 所有数据冒险都能通过调整指令顺序和插入nop指令解决

■ C

- 对于load-use数据冒险，即后一条指令y需要使用前一条访存指令x从主存中取出的数据，由于指令y 中所需的数据在指令x的访存阶段(MEM)结束时才可使用，即使有转发电路，指令y在执行阶段也无法获得所需的数据，只能通过插入nop指令或调整指令顺序解决。

假定某计算机工程师想设计一个新CPU，其中运行的一个典型程序的核心模块需要执行一百万条指令，每条指令执行时间为100ps

- 1) 在非流水线处理器上执行该程序大约需要花多长时间？**
- 2) 若新CPU是一个20级流水线处理器，执行上述同样的程序，理想情况下，它比非流水线处理器快多少？**
- 3) 实际流水线并不是理想的，流水段之间的数据传送会有额外开销。这些开销是否会影响指令执行时间和指令吞吐率？**

- 1) 非流水线处理器上执行该程序的时间大约为 $100\text{ps} \times 10^6 = 100\mu\text{s}$
- 2) 若在一个20级流水线的处理器上执行，忽略流水段之间的寄存器延时，理想情况下，每个时钟周期为 $100/20 = 5\text{ps}$ ，所以，程序执行时间大约为 $5 \times 10^6 = 5\mu\text{s}$ ，因此，大约快 $100/5 = 20$ 倍。
- 3) 流水段之间数据的传递产生的额外开销，使得一条指令的执行时间被延长，即影响了指令执行时间；同时也延长了每个流水段的时间，即影响了指令的吞吐率。

THANKS